COMSM1201 : Exercises in C

Neill Campbell

School of Computer Science, University of Bristol

Copyright © 2026 Neill Campbell

Formatted in LaTeX, based on the Legrand Orange Book from BOOK-WEBSITE.COM

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Contents

1 Hello World 5
1.1 Lecture Notes Chapter B 5
1.2 Twice the Sum 5
1.3 Letter C 5
1.4 Lecture Notes Chapter C 6
1.5 ++a++ 6
1.6 Randomness 6
1.7 Lecture Notes Chapter D 7
1.8 find_max 7
1.9 Loving Oddness 7
1.10 Linear Congruent Generator 7
1.11 Higher-Lower 8
1.12 Cash Machine (ATM) 8
1.13 Triangle Numbers 8
1.14 Lecture Notes Chapter E 9
1.15 Hailstone 9
1.16 Hailstone Sequence 9
1.17 Primes 10
1.18 Triangles 10
1.19 Time Flies 10
1.20 Dirichlet Distribution 11

2 Mathematics & Characters 13
2.1 Lecture Notes Chapter F 13
2.2 Unit Circle 13
2.3 Monte Carlo Π 13
2.4 Leibniz Π 14
2.5 Irrational numbers 14
2.6 Fibonacci Words Using ϕ 14
2.7 Vowelness 15
2.8 Planet Trium 15
2.9 Planet Bob 16
2.10 Secret Codes 17
2.11 Lecture Notes Chapter G 17
2.12 Roulette 17

3 *1D* Arrays & Strings 19
3.1 Lecture Notes Chapter H 19
3.2 Neill's Microwave 19
3.3 Music Playlisters 20
3.4 Yahtzee 20
3.5 Rule 110 20
3.6 The Devil's Dartboard 22
3.7 Counting Sort 23
3.8 Fibonacci Words using Substitution Rules 23
3.9 Lecture Notes Chapter I 23
3.10 Palindromes 24
3.11 Int to String 24
3.12 Roman Numerals 24
3.13 Soundex Coding 25
3.14 Fibonacci Words using Strings 26
3.15 Merging Strings 27
3.16 Rot18 27
3.17 Wordle 27

4 *2D* Arrays 29
4.1 Crosswords 29
4.2 Binary Grid Puzzle 30
4.3 The Game of Life 32

4.4 Life Wars 33
4.5 Wireworld 35
4.6 Forest Fire 35
4.7 Diffusion Limited Aggregation 36
4.8 Langton's Ant 37
4.9 Minesweeper 38

5 Files, argc and Graphics 41
5.1 Anagrams 41
5.2 ANSI Escape Sequences 42
5.3 SDL - Intro 43
5.4 A Simple Spelling Checker 43
5.5 Crush It! 44
5.6 Tents & Trees 45
5.7 Crowns 47

7 Recursion 51
7.1 Word Ladders 51
7.2 Maze 52
7.3 Draw to Unlock 54
7.4 Sierpinski Carpet 55
7.5 Sierpinski Squares 56
7.6 Prime Factors 56

8 Searching Boards 59
8.1 Conway's Soldiers 59
8.2 The 8-Tile Puzzle 60
8.3 Happy Bookcases 60
8.4 Roller-Board 63
8.5 Car Park 67
8.6 The N-Queens Puzzle 69
8.7 Match Drop 72
8.8 NumMatch 73

9 ADTs & Data Structures I 77
9.1 Indexed Arrays 77
9.2 Packed Boolean Arrays 77

9.3 Sets 78
9.4 Towards Polymorphism 78
9.5 Self-Organising Linked Lists 78
9.6 Sudoku 79
9.7 MultiValue Maps 80
9.8 Rhymes 81
9.9 Advent of Code 82

10 Trees & Hashing 83
10.1 Depth 83
10.2 Two Trees 85
10.3 Binary Tree Visualisation 85
10.4 Lowest Common Ancestor 86
10.5 Huffman Encoding 86
10.6 Faster MVMs 87
10.7 Double Hashing 88
10.8 Separate Chaining 88

11 ADTs II & Data Structures 89
11.1 Compressed Sparse Arrays 89
11.2 27-Way Trees 91
11.3 Polymorphic Hashing 93
11.4 Cuckoo Hashing 93
11.5 Exact and Approximate (Bloom) Dictionaries 94
11.6 Binary Sparse Arrays 95
11.7 Cons, Car and Cdr 97

A House Style 101
A.1 Correctness 101
A.2 Prettifying 103
A.3 Readability 104

Lecture Notes Chapter B
Twice the Sum
Letter C
Lecture Notes Chapter C
++a++
Randomness
Lecture Notes Chapter D
find_max
Loving Oddness
Linear Congruent Generator
Higher-Lower
Cash Machine (ATM)
Triangle Numbers
Lecture Notes Chapter E
Hailstone
Hailstone Sequence
Primes
Triangles
Dirichlet Distribution



1. Hello World

Some of the exercises in this Chapter are taken from the book "C by Dissection".

1.1 Lecture Notes Chapter B

Exercise 1.1.1 After you've studied Chapter B (*Hello World!*), compile and run the examples given in the lecture notes. ■

1.2 Twice the Sum

Here is part of a program that begins by asking the user to input three integers:

```
#include <stdio.h>

int main(void)
{
    int a, b, c;

    printf("Input three integers: ");
```

...

Exercise 1.2.1 Complete the program so that when the user executes it and types in 2, 3, and 7, this is what appears on the screen:

```
Input three integers: 2 3 7
Twice the sum of integers plus 7 is 31 !
```

■

1.3 Letter C

Execute this program so you understand the output:

```
#include <stdio.h>
```

```
#define HEIGHT 17

int main(void)
{

    int i = 0;

    printf("\n\nIIIIIII\n");
    while(i < HEIGHT){
        printf(" III\n");
        i = i + 1;
    }
    printf("IIIIIII\n\n\n");
    return 0;

}
```

Exercise 1.3.1 Write a similar program that prints a large letter C on the screen (it doesn't need to be curved!). ■

1.4 Lecture Notes Chapter C

Exercise 1.4.1 After you've studied Chapter C (*Grammar*), compile and run the examples given in the lecture notes. ■

1.5 ++a++

Study the following code and write down what you think it prints.

```
int a, b = 0, c = 0;
a = ++b + ++c;
printf("%i %i %i\n", a, b, c);
a = b++ + c++;
printf("%i %i %i\n", a, b, c);
a = ++b + c++;
printf("%i %i %i\n", a, b, c);
a = b-- + --c;
printf("%i %i %i\n", a, b, c);
```

Exercise 1.5.1 Then a program to check your answers. ■

1.6 Randomness

The function `rand()` returns values in the interval [0, RAND_MAX]. If we declare the variable `median` and initialise it to have the value `RAND_MAX/2`, then `rand()` will return a value that is sometimes larger than `median` and sometimes smaller.

Exercise 1.6.1 Write a program that calls `rand()`, say 500 times, inside a `for` loop, increments the variable `minus_cnt` every time `rand()` returns a value less than `median`. Each time through the `for` loop, print out the value of the difference of `plus_cnt` and `minus_cnt`. You might think that this difference should oscillate near zero. Does it ? ■

‘

1.7 Lecture Notes Chapter D

Exercise 1.7.1 After you've studied Chapter D (*Flow Control*), compile and run the examples given in the lecture notes. ■

1.8 find_max

Exercise 1.8.1 Write a program that finds the largest number entered by the user. Executing the program will produce something like:

```
How many numbers do you wish to enter ? 5
Enter 5 real numbers: 1.01 -3 2.2 7.0700 5
Maximum value: 7.07
```

■

1.9 Loving Oddness

Suppose that you hate even integers but love odd ones.

Exercise 1.9.1 Modify the program you wrote for Exercise 1.8.1 so that all variables are of type `int`, even numbers are ignored and only odd integers are processed. Explain all this to the user via appropriate `printf()` statements. ■

1.10 Linear Congruent Generator

One simple way to generate 'random' numbers is via a Linear Congruent Generator which might look something like this:

```
int seed = 0;
/* Linear Congruential Generator */
for(i=0; i<LOOPS; i++){
    seed = (A*seed + C) % M;
    /* Seed now contains your new random number */
}
```

here, A, C and M are constants defined in the code. All of these pseudo-generators have a period of repetition that is shorter than M. For instance, with A set to 9, C set to 5 and M set to 11 the sequence of numbers is :

```
5
6
4
8
0
5
```

and so repeats after 5 numbers (period equals 5).

Exercise 1.10.1 Adapt the above program so that it prints the period of the LCG, where you've `#defined` the constants A, C and M and `seed` always begins at zero. For the constants described above, the program would output 5. For $A = 7$, $C = 5$ and $M = 11$ it will output 10. ■

1.11 Higher-Lower

In the game "higher-Lower", a user has to guess a secret number chosen by another. They then repeatedly guess the number, being only told whether their guess was greater, or less than the secret one.

Exercise 1.11.1 Write a program that selects a random number between 1 and 1000. The user is asked to guess this number. If this guess is correct, the user is told that they have chosen the correct number and the game ends. Otherwise, they are told if their guess was too high or too low. The user has 10 goes to guess correctly before the game ends and they lose. ■

1.12 Cash Machine (ATM)

Some cash dispensers only contain £20 notes. When a user types in how much money they'd like to be given, you need to check that the amount requested can be dispensed exactly using only £20 notes. If not, a choice of the two closest (one lower, one higher) amounts is presented.

Exercise 1.12.1 Write a program that inputs a number from the user and then prompts them for a better choice if it is not correct. For example :

```
How much money would you like ? 175
I can give you 160 or 180, try again.
How much money would you like ? 180
OK, dispensing ...
```

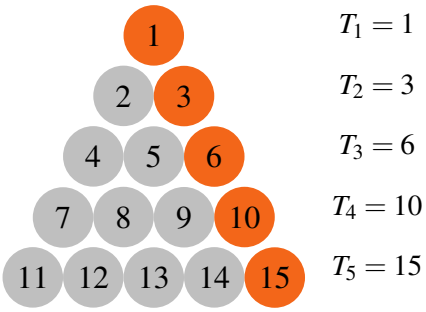
or :

```
How much money would you like ? 25
I can give you 20 or 40, try again.
How much money would you like ? 45
I can give you 40 or 60, try again.
How much money would you like ? 80
OK, dispensing ...
```

In this assessment you may assume the input from the user is "sensible" i.e. is not a negative number etc. ■

1.13 Triangle Numbers

A Triangle number is the sum of numbers from 1 to n. The 5th Triangle number is the sum of numbers $1, 2, 3, 4, 5$, that is 15. They also relate to the number of circles you could stack up as equilateral triangles



Exercise 1.13.1 Write a program that prints out the sequence of Triangle numbers, using iteration, computing the next number based upon the previous.

Check these against:



`http://oeis.org/A000217`

and also by generating the nth Triangle number based on the Equation :

$$T_n = n * (n+1)/2$$

■

1.14 Lecture Notes Chapter E

Exercise 1.14.1 After you've studied Chapter E (*Functions*), compile and run the examples given in the lecture notes. ■

1.15 Hailstone

The next number in a hailstone sequence is $n/2$ if the current number n is even, or $3n+1$ if the current number is odd.

So, for instance, if the initial number is 77, then the following sequence is produced:

```
77
232
116
58
29
88
44
22
11
34
```

Exercise 1.15.1 Write a program that, given a number typed by the user, prints out the sequence of *hailstone* numbers. The sequence terminates when it gets to 1. ■

1.16 Hailstone Sequence

Hailstones sequences are ones that seem to always return to 1. The number is halved if even, and if odd then the next becomes `3*n+1`. For instance, when we start with the number 6, we get the sequence : `6, 3, 10, 5, 16, 8, 4, 2, 1` that has nine numbers in it. When we start with the number 11, the sequence is longer, containing 15 numbers : `11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1.`

Note that whilst the sequences tend to be relatively short, hailstone numbers can be extremely large. You may need a storage type larger than an *int* for storing them.

Exercise 1.16.1 Write a program that :

- displays which initial number (less than $10,000,000$) creates the **longest** hailstone

sequence.

■

Exercise 1.16.2 Write a program that :

- displays which initial number (less than $10,000,000$) leads to the **largest** number appearing in the sequence.

■

1.17 Primes

A prime number can only be exactly divided by itself or 1. The number 17 is prime, but 16 is not because the numbers 2, 4 and 8 can divide it exactly. (Hint $16\%4 == 0$).

Exercise 1.17.1 Write a program that prints out the first n primes, where n is input by the user. The first 8 primes are:

```
2
3
5
7
11
13
17
19
```

What is the 3000th prime ? ■

1.18 Triangles

A triangle can be equilateral (all three sides have the same length), isosceles (has two equal length sides), scalene (all the sides have a different length), or right angled where if the three sides are a, b and c, and c is the longest, then : $c = \sqrt{a^2 + b^2}$

Exercise 1.18.1 Write a program so that you can process a number of triples of side lengths in a single run of your program using a suitable unlikely input value for the first integer in order to terminate the program. e.g. -999.

Think hard about the test data for your program to ensure that all possible cases are covered and all invalid data results in a sensible error message. Such cases can include sides of negative length, and impossible triangles (e.g. one side is longer than the sum of the other two). ■

1.19 Time Flies

Exercise 1.19.1 Write a program which allows the user to enter two times in 24-hour clock format, and computes the length of time between the two, e.g.:

```
Enter two times : 23:00 04:15
Difference is : 5:15
```

or,

```
Enter two times : 23:40 22:50
```

```
Difference is : 23:10
```

■

1.20 Dirichlet Distribution

When you first start thinking about prime numbers, it becomes pretty obvious that most of them end with the digit $1, 3, 7$ or 9 (they can't end with two, because they can't be even etc.)

Exercise 1.20.1 Write a program that computes what fraction of all primes end with a 3. You can do this by generating a large number of primes, in sequence, and keeping a running count of the number that end with 3, as compared to the total of primes generated. ■

Lecture Notes Chapter F
Unit Circle
Monte Carlo Π
Leibniz Π
Irrational numbers
Fibonacci Words Using ϕ
Vowelness
Planet Trium
Planet Bob
Secret Codes
Lecture Notes Chapter G
Roulette



2. Mathematics & Characters

Note that these exercises are in **NO** particular order - try the ones you find more straightforwards before attempting complex ones. Remember to assert `test()` all of your functions.

2.1 Lecture Notes Chapter F

Exercise 2.1.1 After you've studied Chapter F (*Mathematics & Characters*), compile and run the examples given in the lecture notes. ■

2.2 Unit Circle

In mathematics, for all (real) x, it is true that:

$$sin^2(x) + cos^2(x) = 1$$

i.e. $sin(x) * sin(x) + cos(x) * cos(x) = 1$.

Exercise 2.2.1 Write a program to demonstrate this for values of x input by the user. ■

2.3 Monte Carlo Π

At :

www `https://www.geeksforgeeks.org/estimating-value-pi-using-monte-carlo`

a square whose sides are of length r, and a quarter-circle, whose radius is of r are drawn.

If you throw random darts at the square, then many, but not all, also hit the circle. A dart landing at position (x,y) only hits the circle if $x^2 + y^2 \leq r^2$.

The area of the circle is $\frac{\pi}{4}r^2$, and the area of the square is r^2.

Therefore, a way to approximate π, is to choose random (x,y) pairs inside the square h_a, and count the h_c ones that hit the circle. Then:

$$\pi \approx \frac{4h_c}{h_a} \tag{2.1}$$

Exercise 2.3.1 Write a program to run this simulation, and display the improving version of the approximation to π. ■

2.4 Leibniz Π

See:

www `https://en.wikipedia.org/wiki/Leibniz_formula_for_%CF%80`

The Mathematical constant π can be approximated using the formula :

$$\pi = 4 - \tfrac{4}{3} + \tfrac{4}{5} - \tfrac{4}{7} + \tfrac{4}{9} - \tfrac{4}{11} + \ldots$$

Notice the pattern here of alternating $+$ and $-$ signs, and the odd divisors.

Exercise 2.4.1 Write a program that computes π looping through smaller and smaller fractions of the series above. How many iterations does it take to get π correctly approximated to 7 decimal places ? ■

2.5 Irrational numbers

Neill's favourite number (don't ask why!) is e which has the value $2.71828182845904523536\ldots$ This is an example of an irrational number - one that can only ever be *approximated* by the ratio of two integers - a poor approximation of e is 87 divided by 32, that is $\frac{87}{32}$.

Exercise 2.5.1 Write a program that loops through all possible denominators (that is the integer on the bottom, b in the fraction $\frac{a}{b}$) and finds which a and b pair give the best approximation to e. The output of your program might look something like :

```
271801/99990 = 2.71828182818281849364
```

You need only investigate denominators $< 100,000$. #define the number being searched for. Check your code works for other famous constants such as $\pi = 3.14159265358979323846\ldots$:

```
312689/99532 = 3.14159265361893647039
```

■

2.6 Fibonacci Words Using ϕ

www `https://en.wikipedia.org/wiki/Fibonacci_word`

A *Fibonacci Word* is simply a particular (infinite) sequence of digits:

`0100101001001010010100100101001001010010100100101001010010010100101001001010010010...`

and shouldn't be confused with the related, but totally different *Fibonacci Sequence* (which we'll see later in the lecture series).

You start by defining S_0 to be the sequence "0", and S_1 to be "01". Each subsequent part of the sequence is made by simply concatenating the previous two, so $S_2 = 010$ and $S_3 = 01001$ and so on.

Perhaps somewhat surprisingly, the nth-digit can be predicted without computing the rest of the sequence, using the equation :

$$2 + \lfloor n\phi \rfloor - \lfloor (n+1)\phi \rfloor \tag{2.2}$$

or, maybe more understandably :

$$2 + \text{floor}(n\phi) - \text{floor}((n+1)\phi) \tag{2.3}$$

Here `floor()` is the function that rounds a number down to its lowest integer part (e.g. 1.815 becomes 1.0), and where ϕ is the golden ratio $= 1.61803398875...$. Note also that the sequence is assumed to begin at the $n = 1$, and not at zero which a programmer might assume!

Exercise 2.6.1 Write the function:

```
bool fibword_phi(int n)
```

which returns to n^{th}-digit of the sequence (which begins at $n = 1$) using: an approximation of ϕ, the function `floor()` found in the mathematics library, and Equation 2.3. ■

[Since ϕ can only be stored approximately on a computer, for larger numbers of n, this equation doesn't hold. Exactly where this happens depends upon how closely you have approximated ϕ. We will eplore this later in Exercise 3.8.2.]

2.7 Vowelness

Vowels are the letters *a*, *e*, *i*, *o* and *u*.

Exercise 2.7.1 Write a program that reads characters from the keyboard and writes to the screen. Write all vowels as uppercase letters, and all non-vowels as lowercase letters. Do this by using writing a function `isvowel()` that tests whether or not a character is a vowel. ■

2.8 Planet Trium

On the planet Trium, everyone's name has three letters. Not only this, all the names take the form of non-vowel, vowel, non-vowel.

Exercise 2.8.1 Write a program that outputs all the valid names and numbers them. The first few should look like :

```
1 bab
2 bac
3 bad
4 baf
5 bag
6 bah
7 baj
8 bak
```

```
 9 bal
10 bam
11 ban
12 bap
13 baq
14 bar
15 bas
16 bat
17 bav
18 baw
19 bax
20 bay
21 baz
22 beb
23 bec
24 bed
25 bef
26 beg
```

2.9 Planet Bob

On the planet Bob, everyone's name has three letters. These names either take the form of consonant-vowel-consonant or else vowel-consonant-vowel. For the purposes here, vowels are the letters `{a,e,i,o,u}` and consonants are all other letters. There are two other rules :

1. The first letter and third letters of the name must always be the same.
2. The name is only 'valid' if, when you sum up the values of the three letters ($a = 1, b = 2$ etc.), the sum is prime.

The name "bob" is a valid name: it has the form consonant-vowel-consonant, the first letter and third letters are the same ('b') and the three letters sum to $19(2+15+2)$, which is prime. The name "aba' is **not** valid, since the sum of the three letters is $4(1+2+1)$ which is **not** prime.

Exercise 2.9.1 Write a program that outputs all the valid names and numbers them. The first few names should look like :

```
 1 aca
 2 aka
 3 aqa
 4 bab
 5 bib
 6 bob
 7 cac
 8 cec
 9 ded
10 did
11 dod
12 dud
13 ece
14 ege
15 eme
16 ese
```

```
17 faf
```

■

2.10 Secret Codes

Write a program that converts a stream of text typed by the user into a 'secret' code. This is achieved by turning every letter 'a' into a 'z', every letter 'b' into a 'y', every letter 'c' into and 'x' and so on.

Exercise 2.10.1 Write a program which includes a function whose 'top-line' is :

```
int secret(int a)
```

that takes a character, and returns the secret code for this character. Note that the function **does** need to preserve the case of the letter, and that non-letters are returned unaffected.

When the program is run, the following input:

```
The Quick Brown Fox Jumps Over the Lazy Dog !
```

produces the following output :

```
Gsv Jfrxp Yildm Ulc Qfnkh Levi gsv Ozab Wlt !
```

■

2.11 Lecture Notes Chapter G

Exercise 2.11.1 After you've studied Chapter G (*Prettifying*), compile and run the examples given in the lecture notes. ■

2.12 Roulette

This is an chance for you to practice self-document techniques such as sensible identifier naming, commenting, `typedefs` and enumeration.

Exercise 2.12.1 Write a roulette program. The roulette machine will select a number between 0 and 35 at random. The player can place an odd/even bet, or a bet on a particular number. A winning odd/even bet is paid off at 2 to 1, except that all odd/even bets lose if the roulette selects 0. If the player places a bet on a particular number, and the roulette selects it, the player is paid off at 35 to 1. ■

Lecture Notes Chapter H
Neill's Microwave
Music Playlisters
Yahtzee
Rule 110
The Devil's Dartboard
Counting Sort
Fibonacci Words using Substitution Rules
Lecture Notes Chapter I
Palindromes
Int to String
Roman Numerals
Soundex Coding
Fibonacci Words using Strings
Merging Strings
Rot18
Wordle



3. 1*D* Arrays & Strings

Note that these exercises are in **NO** particular order - try the ones you find more straightforwards before attempting complex ones. Remember to assert `test()` all of your functions.

3.1 Lecture Notes Chapter H

Exercise 3.1.1 After you've studied Chapter H (1*D Arrays*), compile and run the examples given in the lecture notes. ■

3.2 Neill's Microwave

Last week I purchased a new, state-of-the-art microwave oven. To select how long you wish to cook food for, there are three buttons: one marked "10 minutes", one marked "1 minute" and one marked "10 seconds". To cook something for 90 seconds requires you to press the "1 minute" button, and the "10 seconds" button three times. This is four button presses in total. To cook something for 25 seconds requires three button presses; the "10 second" button needs to be pressed three times and we have to accept a minor overcooking of the food.

Exercise 3.2.1 Using an array to store the cooking times for the buttons, write a program that, given a required cooking time in seconds, allows the minimum number of button presses to be determined.

Example executions of the program will look like :

```
Type the time required
25
Number of button presses = 3
Type the time required
705
Number of button presses = 7
```

■

3.3 Music Playlisters

Most MP3 players have a "random" or "shuffle" feature. The problem with these is that they can sometimes be **too** random; a particular song could be played twice in succession if the new song to play is truly chosen randomly each time without taking into account what has already been played.

To solve this, many of them randomly order the entire playlist so that each song appears in a random place, but once only. The output might look something this:

```
How many songs required ? 5
4 3 5 1 2
```

or :

```
How many songs required ? 10
1 9 10 2 4 7 3 6 5 8
```

Exercise 3.3.1 Write a program that gets a number from the user (to represent the number of songs required) and outputs a randomised list. ■

Exercise 3.3.2 Rewrite Exercise 3.3.1 so that the program passes an array of integers (e.g. `[1,2,3,4,5,6,7,8,9,10]`) to a function which shuffles them **in-place** (no other arrays are used) and with an algorithm having complexity $O(n)$. ■

3.4 Yahtzee

The game of Yahtzee is a game played with five dice, and you try to obtain certain 'hands'. In a similar way to poker, these hands could include a *Full House* (two dice are the same, and another three are the same), e.g.:

⚃⚃⚀⚃⚀ or ⚄⚀⚀⚀⚄

or another possible hand is *Four-of-a-Kind*, e.g.: ⚂⚂⚀⚂⚂ or ⚅⚅⚅⚀⚅ (but not ⚂⚂⚂⚂⚂ which is *Five-of-a-Kind*)

A little mathematics tells use that the probability of these two hands should be 3.85% and 1.93% respectively.

Exercise 3.4.1 Complete the program `yahtzee.c` (which is in the usual place online), by analysing a large number of random dice rolls, the probabilty of each of these two hands. The five dice of the hand are stored in an array, and to facilitate deciding which hand you've got, a histogram is computed to say how often a ⚀ occurs in the hand, how often a ⚁ occurs and so one. A *Full-House* occurs when both a 2 and a 3 occurs in the histogram; a *Four-of-a-Kind* occurs when there is a 4 somewhere in the histogram.

■

3.5 Rule 110

Rather interesting patterns can be created using *Cellular Automata*. Here we will use a simple example, one known as *Rule 110* : The idea is that in a 1D array, cells can be either on ■ or off □ (perhaps represented by the integer values 1 and 0). A new 1D array is created in which we decide upon the state of each cell in the array based on the cell above and its two immediate neighbours.

If the three cells above are all ‘on’, then the cell is set to ‘off’ ($111 \rightarrow 0$). If the three cells above are ‘on’, ‘on’, ‘off’ then the new cell is set to ‘on’ ($110 \rightarrow 1$). The rules, in full, are:
$111 \rightarrow 0$
$110 \rightarrow 1$
$101 \rightarrow 1$
$100 \rightarrow 0$
$011 \rightarrow 1$
$010 \rightarrow 1$
$001 \rightarrow 1$
$000 \rightarrow 0$

You take a 1D array, filled with zeroes or ones, and based on these, you create a new 1D array of zeroes and ones. Any particular cell uses the three cells ‘above’ it to make the decision about its value. If the first line has all zeroes and a single one in the middle, then the automata evolves as:

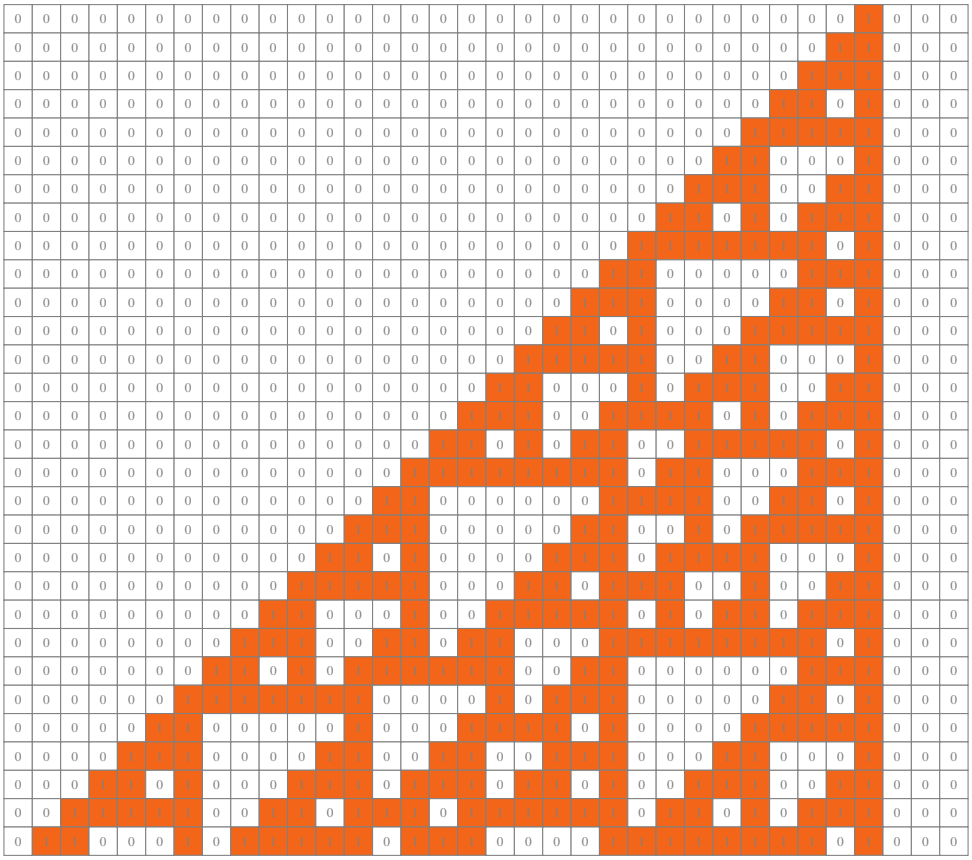


Figure 3.1: 1D cellular automaton using Rule 110. Top line shows initial state, each subsequent line is produced from the line above it. Each cell has a rule to switch it ‘on’ or ’off’ based on the state of the three cells above it in the diagram.

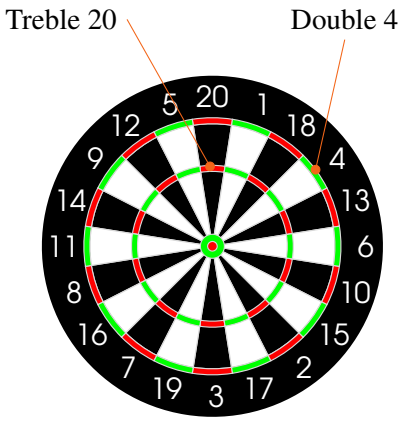
Exercise 3.5.1 Write a program that outputs something similar to Figure 3.1, but using plain text, giving the user the option to start with a randomised first line, or a line with a single ‘on’ in the central location. Do not use $2D$ arrays for this - a couple of $1D$ arrays is sufficient. ■

Exercise 3.5.2 Rewrite the program above to allow other rules to be displayed - for instance 124, 30 and 90.

www http://en.wikipedia.org/wiki/Rule_110

■

3.6 The Devil's Dartboard



In the traditional 'pub' game, darts, there are 62 different possible scores : single 1 - 20 (the white and black areas), double 1 - 20 (the outer red and green segments) (i.e. 2, 4, 6, 8 ...), treble 1 - 20 (i.e. 3, 6, 9, 12 ...) (the inner red or green segments), 25 (small green circle) and 50 (the small red inner circle).

It's not obvious, if you were inventing darts from scratch, how best to lay out the numbers. The London board shown seems to have small numbers near high numbers, so that if you just miss the 20 for example, then you'll hit a small number instead.

Here we look at a measure for the 'difficulty' of a dartboard. One approach is to simply sum up the values of adjacent triples, squaring this number. So for the London board shown, this would be: $(20+1+18)^2+(1+18+4)^2+(18+4+13)^2\ldots(5+20+1)^2 = 20478$

For our purposes a **lower** number is better[1]. For more details see :

www http://www.mathpages.com/home/kmath025.htm

Exercise 3.6.1 Write a program that repeatedly chooses two positions on the board at random and swaps them. If this leads to a lower cost, keep the board. If not, unswap them. Repeat this *greedy search* 5000000 times, and print out the best board found. Begin with the trivial monotonic sequence. The output may look something like :

```
Total = 19966 :  3 19 11  2 18 12  1 20 10  4 16  8
14  5 13 15  6  7 17  9
```

or

```
Total = 19910 :  3 18 10  5 16  9  8 14 11  4 19  6
```

[1] It's beyond the scope here to explain why!

```
7 20  2 13 15  1 17 12
```

Is the score of 19874 the lowest possible that may be obtained via this technique ? ■

3.7 Counting Sort



UNDER CONSTRUCTION - Please don't attempt this yet.
We'll have this new exciting exercise ready for you soon!

3.8 Fibonacci Words using Substitution Rules

This follows on from Exercise 2.6.1, where we saw a good approximation of the Fibonacci Word using ϕ.

To be certain of the sequence at very large values of n, you'd need to construct the sequence iteratively. One interesting way of doing this is to use a Substitution Rule. Start with a counter indexing a single digit zero at the beginning of a `bool` array. Now at each step, if the digit is 0, append a 1 and a 0 onto the end of the array. If the digit is a 1 append a 0 onto the end of the array. Either way, move along to the next digit and repeat.

```
Array initially :
0 (index zero contains a zero therefore append 1 0)
010 (index 1 contains a one therefore append 0)
0100 (index 2 contains a zero therefore append 1 0)
010010 (index 3 ...)
```

Exercise 3.8.1 Write the function:

```
bool fibword_subs(int n)
```

which returns to n^{th}-digit of the sequence (which begins at $n = 1$) using an array of `bools`, and repeatedly applying the substitution rules described above, until you have a sequence large enough to extract the nth-digit. ■

Exercise 3.8.2 Write a program which experiments with the approximation of ϕ from Exercise 2.6.1, to see at what value of n it begins to return incorrect values. These 'correct' values can be obtained from the function written in Exercise 3.8.1.

In my implementation, when I

```
#define PHI 1.61
```

the values are wrong from the 12th digit. For:

```
#define PHI 1.61803
```

the values are wrong from the 609th digit.

■

3.9 Lecture Notes Chapter I

Exercise 3.9.1 After you've studied Chapter I (*Strings*), compile and run the examples given in the lecture notes. ■

3.10 Palindromes

From `wikipedia.org` :

> A palindrome is a word, phrase, number or other sequence of units that has the property of reading the same in either direction (the adjustment of punctuation and spaces between words is generally permitted).
>
> The most familiar palindromes, in English at least, are character-by-character: the written characters read the same backwards as forwards. Palindromes may consist of a single word (such as "civic" or "level"), a phrase or sentence ("Neil, a trap! Sid is part alien!", "Was it a rat I saw?") or a longer passage of text ("Sit on a potato pan, Otis."), even a fragmented sentence ("A man, a plan, a canal: Panama!", "No Roman a moron"). Spaces, punctuation and case are usually ignored, even in terms of abbreviation ("Mr. Owl ate my metal worm").

Exercise 3.10.1 Write a program that prompts a user for a phrase and tells them whether it is a palindrome or not. **Do not** use any of the built-in string-handling functions (`string.h`), such as `strlen()` and `strcmp()`. However, you **may** use the character functions (`ctype.h`), such as `islower()` and `isalpha()`.

Test your program with (amongst others) the following palindromes :

```
"kayak"
"A man, a plan, a canal: Panama!"
"Madam, in Eden I'm Adam,"
"Level, madam, level!"
```

■

3.11 Int to String

Exercise 3.11.1 Write a function that converts an integer to a string, using the skeleton code in the usual place online. works correctly:

The integer may be signed (i.e. be positive or negative) and you may assume it is in base-10. Avoid using any of the built-in string-handling functions to do this (e.g. `snprintf()`) including all those in `string.h`. ■

3.12 Roman Numerals

Adapted from:



`http://mathworld.wolfram.com/RomanNumerals.html`

> "Roman numerals are a system of numerical notations used by the Romans. They are an additive (and subtractive) system in which letters are used to denote certain "base" numbers, and arbitrary numbers are then denoted using combinations of symbols. Unfortunately, little is known about the origin of the Roman numeral system.

The following table gives the Latin letters used in Roman numerals and the corresponding numerical values they represent :

I	1
V	5
X	10
L	50
C	100
D	500
M	1000

For example, the number 1732 would be denoted MDCCXXXII in Roman numerals. However, Roman numerals are not a purely additive number system. In particular, instead of using four symbols to represent a 4, 40, 9, 90, etc. (i.e., IIII, XXXX, VIIII, LXXXX, etc.), such numbers are instead denoted by preceding the symbol for 5, 50, 10, 100, etc., with a symbol indicating subtraction. For example, 4 is denoted IV, 9 as IX, 40 as XL, etc."

It turns out that every number between 1 and 3999 can be represented as a Roman numeral made up of the following one- and two-letter combinations:

I	1	IV	4
V	5	IX	9
X	10	XL	40
L	50	XC	90
C	100	CD	400
D	500	CM	900
M	1000		

Exercise 3.12.1 Write a program that contains a function which is passed a string and returns an integer. The string is a roman numeral in the range `1 - 3999`. Amongst others, `assert()` test that `MCMXCIX` returns `1999`, `MCMLXVII` returns `1967` and that `MCDXCI` returns `1491`. ■

3.13 Soundex Coding

First applied to the 1880 census, Soundex is a phonetic index, not a strictly alphabetical one. Its key feature is that it codes surnames (last names) based on the way a name sounds rather than on how it is spelled. For example, surnames that sound the same but are spelled differently, like Smith and Smyth, have the same code and are indexed together. The intent was to help researchers find a surname quickly even though it may have received different spellings. If a name like Cook, though, is spelled Koch or Faust is Phaust, a search for a different set of Soundex codes and cards based on the variation of the surname's first letter is necessary.

To use Soundex, researchers must first code the surname of the person or family in which they are interested. Every Soundex code consists of a letter and three numbers, such as B536, representing names such as Bender. The letter is always the first letter of the surname, whether it is a vowel or a consonant.

The detailed description of the algorithm may be found at :



`http://www.highprogrammer.com/alan/numbers/soundex.html`

The first letter is simply the first letter in the word. The remaining numbers range from 1 to 6, indicating different categories of sounds created by consonants following the first letter. If the word is too short to generate 3 numbers, 0 is added as needed. If the generated code is longer than 3 numbers, the extra are thrown away.

Code	*Letters Description*
1	*B, F, P, V Labial*
2	*C, G, J, K, Q, S, X, Z Gutterals and sibilants*
3	*D, T Dental*
4	*L Long liquid*
5	*M, N Nasal*
6	*R Short liquid*
SKIP	*A, E, H, I, O, U, W, Y Vowels (and H, W, and Y) are skipped*

There are several special cases when calculating a soundex code:

- *Letters with the same soundex number that are immediately next to each other are discarded. So Pfizer becomes Pizer, Sack becomes Sac, Czar becomes Car, Collins becomes Colins, and Mroczak becomes Mrocak.*
- *If two letters with the same soundex number seperated by "H" or "W", only use the first letter. So Ashcroft is treated as Ashroft.*

Sample Soundex codes:

Word	*Soundex*
Washington	*W252*
Wu	*W000*
DeSmet	*D253*
Gutierrez	*G362*
Pfister	*P236*
Jackson	*J250*
Tymczak	*T522*
Ashcraft	*A261*

Exercise 3.13.1 Write a program that contains a function which is passed a name as a string, and returns the soundex code for it. Using `assert()` tests, check it works correctly for, amongst others, all the examples above.

■

3.14 Fibonacci Words using Strings

See Section 2.6.

Exercise 3.14.1 Write the function:

```
bool fibword_str(int n)
```

which returns to n^{th}-digit of the sequence (which begins at $n = 1$) using strings. Previous strings S_{k-1} and S_{k-2} are repeatedly concatenated to create the new S_k, until you have a sequence large enough to extract the nth-digit.

To achieve this, use fixed-size strings that are set to always be large enough. ■

3.15 Merging Strings



Experience shows that the next exercise is trickier than most - you have been warned!

Exercise 3.15.1 Write the function `strmerge()` which concatenates `s1` and `s2` into `s3`, discarding any overlapping characters which are common to the end (tail) of `s1` and the beginning (head) of `s2`. To help, I've put some skeleton code in the usual place online.

Hint : Functions such as `strncmp()` and `strcat` might be useful. ■

3.16 Rot18

ROT18 is an encryption algorithm made of the combination of ROT13 and ROT5. ROT13 replaces each letter in a string with the letter 13 places later in the alphabet, wrapping back to the beginning of the alphabet where necessary e.g. 'a' becomes 'n', 'b' becomes 'o', and 'n' becomes 'a'. ROT5 replaces each digit in a string with the digit 5 greater, wrapping back to 0 where necessary e.g. '0' becomes '5', '1' becomes '6', and '5' becomes '0'.



`https://www.boxentriq.com/code-breaking/rot13`

Exercise 3.16.1 Write the function whose "top-line" is:

void rot(**char** str[])

that takes a string as input and encrypts it using the ROT18 cipher. The function must preserve the case of any letters, and any characters that aren't letters or digits must be unaffected.

When the string `"Hello, World!"` is encrypted, it becomes `"Uryyb, Jbeyq!"` and when encrypted a second time, reverts to the original string.

```
Erzrzore:  fubeg shapgvbaf; ubhfr-fglyr ehyrf :-)
```

■

3.17 Wordle

Online puzzles have grown in popularity, and one of the most widely played examples is *Wordle*:



`https://www.nytimes.com/games/wordle/index.html`

Wordle is a word-guessing game built around comparisons between a **guess word** and the hidden **answer word**, both of which are five letters long. Each letter in the guess is checked against the answer, and feedback is given in three possible ways:

Green: the guessed letter matches the answer letter in the same position.
Yellow: the guessed letter is present somewhere in the answer but not in the correct position.
Unmarked: the guessed letter does not appear in the answer at all.

A key detail is how repeated letters are treated. Suppose the hidden answer is `WORLD` and the guess is `HELLO`:

W	O	R	L	D
H	E	L	L	O

- The `H` in position 0 is unmarked.
- The `E` in position 1 is unmarked.
- The `L` in position 2 is unmarked, as the single `L` in the answer has already been "used up" by a green match.
- The `L` in position 3 is green, as it matches the `L` in the same position of the answer.
- The `O` in position 4 is yellow, as it matches the `O` in position 1 of the answer.

For another example, suppose the hidden answer is `APPLE` and the guess is `ALLEY`:

A	P	P	L	E
A	L	L	E	Y

- The `A` in position 0 is green, as it matches the `A` in the same position of the answer.
- The `L` in position 1 is yellow, because there is an `L` in the answer (position 3).
- The `L` in position 2 is unmarked, as the single `L` in the answer has already been "used up" by the earlier yellow match.
- The `E` in position 3 is yellow, as it matches the `E` in position 4 of the answer.
- The `Y` in position 4 is unmarked.

This avoids double-counting: once an occurrence of a letter in the answer has been matched, it cannot be reused for another match.

Exercise 3.17.1 Code for this exercise can be found in:

www `https://github.com/csnwc/Exercises-In-C/Code/Week3/Wordle`

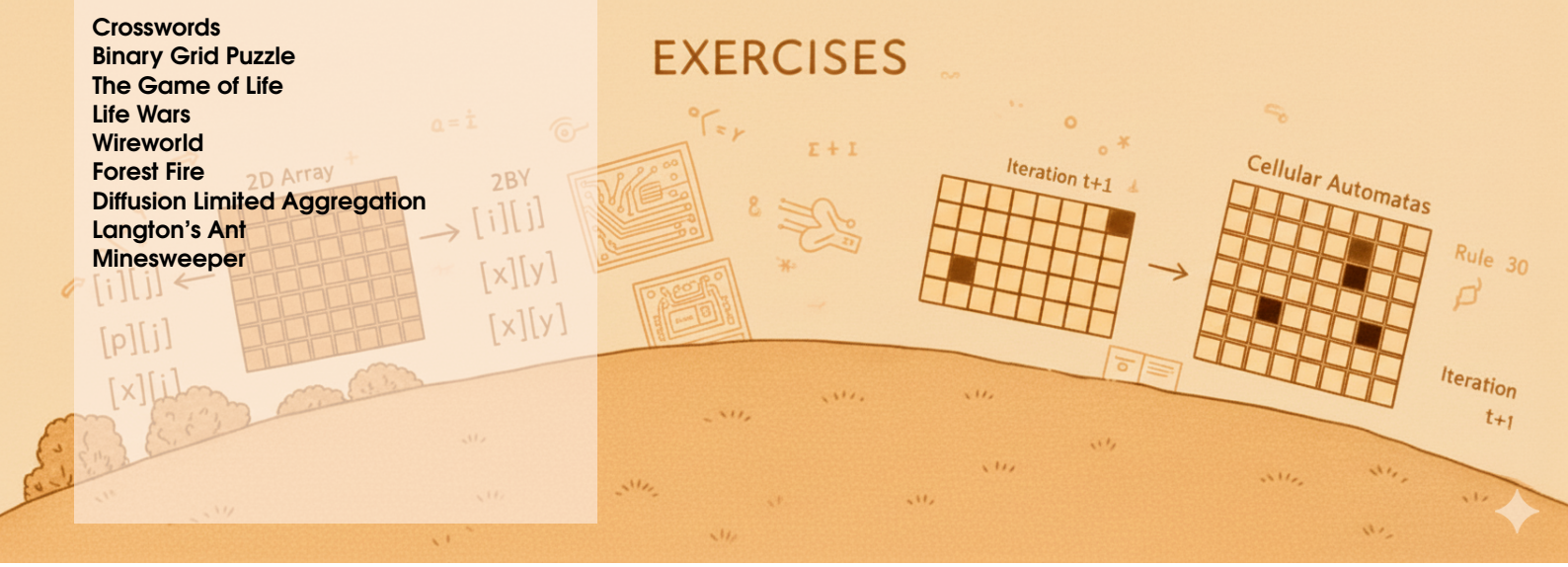
Complete the file **wordle.c**, which together with my files *wordle.h* and *main.c*, allows Wordle guesses to be checked using the case-insensitive functions whose "top-lines" are:

```
int num_greens(char answer[], char guess[])
int num_yellows(char answer[], char guess[])
```

that each take two strings as input and, considering the first string as the answer and the second string as the guess, returns the number of greens/yellows in the guess or -1 if there are any problems.

The file *wordle.c* currently contains standard AI-generated code, which must be edited so it passes any valid assertions. The file *wordle.h* provides the function prototypes you must implement, while *main.c* contains the `main()` function to run the code - a *Makefile* is provided to compile everything. The `test()` function should be used to check all functions your code, including any "helper" functions you may write. Do not alter or resubmit *wordle.h* or *main.c* - my originals will be used. ■

Crosswords
Binary Grid Puzzle
The Game of Life
Life Wars
Wireworld
Forest Fire
Diffusion Limited Aggregation
Langton's Ant
Minesweeper



4. 2D Arrays

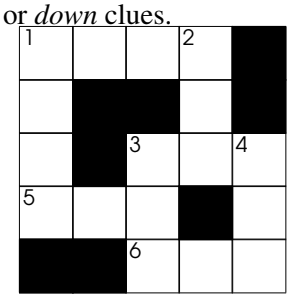
Note that these exercises are in **NO** particular order - try the ones you find more straightforwards before attempting complex ones. Remember to assert `test()` all of your functions.

4.1 Crosswords

Many newspapers in the UK have a puzzles section. One of the most well established such puzzle is the *crossword*.

www `https://en.wikipedia.org/wiki/Crossword`

A crossword is a 2*D* grid of full and empty cells. Clues are given so that the empty squares can be filled-in with answers to *across* or *down* clues.



Across 1 Tailless amphibian 3 Join two items together 5 Pull something suddenly 6 Large deciduous tree

Down 1 Shape having one face and size 2 Higher being 3 Period of History 4 Water barrier

Here though, we aren't going to be dealing with solving crosswords, only in understanding something about their structure.

Firstly, let us consider how crosswords are numbered. In the above example the first square is shared by both 1 across and 1 down. Working from the top-left rightwards and **then downwards**, squares that are at the start of a new word are numbered. The second square in the above example has no number because it is already part of 1 across, and there is no room for a down clue. On the fourth square, there is room for a down clue, but not for an across since it is already being used by 1 down.

Actually, the algorithm for deciding on whether a square should be numbered or not is fairly simple:

1. All squares outside (off the grid) of the board are considered full.
2. The pattern:



shows that we can start a new across clue from the middle square.

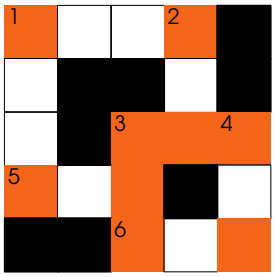
3. Similarly, the pattern:



shows that we can start a new down clue from the middle square.

4. Sometimes a square can be the start of both an across and down clue. In this case, they must share the same number.

Secondly, another important concept in crossword clues is that of the number of *checked* squares. This is the number of squares which are shared by both an across clue and a down clue. In the above crossword, nine squares are shared of the 17 empty ones which rounds to 53%. This is shown below:



Exercise 4.1.1 Complete the file **crossword.c** which, along with my files *crossword.h* and *cwddriver.c*, allows a crossword to be created such that the clue numberings and percentage of checked squares can be found.
My file *crossword.h* contains the function definitions which you'll have to implement in your **crossword.c** file. My file *cwddriver.c* contains the `main()` function to act as a driver to run the code. Your file will contain many other functions as well as those specified, so you'll wish to test them as normal using the `test()` function.
If all of these files are in the same directory, you can compile them using:

```
clang crossword.c cwddriver.c -o crossword -Wall -Wextra
-Wpedantic -std=c99 -Wvla -Wfloat-equal -O3 -Werror -lm
```

Do not alter or resubmit *crossword.h* or *cwddriver.c* - my original versions will be used to compile the *crossword.c* file that you adapt. ■

4.2 Binary Grid Puzzle

Many newspapers in the UK have a puzzles section. One of that has become popular recently is the *Binary Grid*

www `https://www.binarypuzzle.com/`

A Binary Grid is a 2D square grid of cells, each of which is either empty, or contains the value 0 or 1. A set of rules governs what is legal in a **valid** board :

- Each cell should contain either a zero or a one.

- No more than two of the same number can appear consecutively in a row or a column.
- There must be an equal number of 0*s* and 1*s* in each row and column - this means that the height/width of the puzzle must be even.

Given these rules[1] we can deduce some principles to help us find a solution :

Pairs : If two of the same number (0*s* or 1*s*) are in adjacent cells (either horizontally or vertically) then the two surrounding cells on either side must be the other number, otherwise there would be three of the same number in sequence.

OXO : If there are two of the same number with a gap of one empty cell between them (horizontally or vertically), then the opposite number must be in the empty cell (otherwise we'd have three of the same number in sequence).

Counting : If all of a particular number have been found in a row (or column) - that is 3 if the grid is a 6*x*6, then the remaining 3 unknown cells must all contain the other number.

An example puzzle might look so :

·	·	·	·	·	0
0	·	·	1	1	·
·	·	·	1	·	·
·	·	·	·	·	0
·	·	·	·	·	·
·	·	·	·	·	·

By applying the **Pairs** principle 4 times we get :

·	·	·	0	·	0
0	·	0	1	1	0
·	·	·	1	·	·
·	·	·	0	·	0
·	·	·	·	·	·
·	·	·	·	·	·

The **OXO** principle gives :

·	·	·	0	·	0
0	1	0	1	1	0
·	·	·	1	·	·
·	·	·	0	·	0
·	·	·	·	·	·
·	·	·	·	·	·

Application of the Counting principle can now fill the rightmost column :

[1] In the full version of this puzzle, no row or column can be the same - we ignore this constraint here.

·	·	·	0	·	0
0	1	0	1	1	0
·	·	·	1	·	1
·	·	·	0	·	0
·	·	·	·	·	1
·	·	·	·	·	1

Some (but not all) puzzles can be solved by repeatedly applying these three rules. This is what we explore in this assignment.

Exercise 4.2.1 Code for this exercise can be found in

www `https://github.com/csnwc/Exercises-In-C`

then navigate into `Code/Week4/BinaryGrid`.

Complete the file **bingrid.c** which, along with my files *bingrid.h* and *bingrid_driver.c*, allows Binary Grids to be solved.
My file *bingrid.h* contains the function definitions which you'll have to implement in your **bingrid.c** file. My file *bingrid_driver.c* contains the `main()` function to act as a driver to run the code. Your file will contain many other functions as well as those specified, so you'll wish to test them as normal using the `test()` function.
If all of these files are in the same directory, you can compile them using the `Makefile` given. Do not alter or resubmit *bingrid.h* or *bingrid_driver.c* - my original versions will be used to compile the *bingrid.c* file that you adapt.

■

4.3 The Game of Life

The Game of Life was developed by British mathematician John Horton Conway. In Life, a board represents the world and each cell a single location. A cell may be either empty or inhabited. The game has three simple rules, which relate to the cell's eight nearest neighbours :

1. **Survival** An inhabited cell remains inhabited if exactly 2 or 3 of its neighbouring cells are inhabited.
2. **Death** An inhabited cell becomes uninhabited if fewer than 2, or more than 3 of its neighbours are inhabited.
3. **Birth** An uninhabited cell becomes inhabited if exactly 3 of its neighbours are inhabited.

The next board is derived solely from the current one. The current board remains unchanged while computing the next board. In the simple case shown here, the boards alternate infinitely between these two states.

0	0	0	0	0
0	0	1	0	0
0	0	1	0	0
0	0	1	0	0
0	0	0	0	0

0	0	0	0	0
0	0	0	0	0
0	1	1	1	0
0	0	0	0	0
0	0	0	0	0

The 1.06 format

A general purpose way of encoding the input board is called the Life 1.06 format :



http://conwaylife.com/wiki/Life_1.06

This format has comments indicated by a hash in the first column, and the first line is always:

```
#Life 1.06
```

Every line specifies an x and y coordinate of a live cell; such files can be quite long. The coordinates specified are relative to the middle of the board, so :

```
0  -1
```

means the middle row, one cell to the left of the centre.

There are hundreds of interesting patterns stored like this on the above site.

Exercise 4.3.1 Write a program which is run using the `argc` and `argv` parameters to `main`. The usage is as follows :

```
% life file1.lif 10
```

where `file1.lif` is a file specifying the initial state of the board, and `10` specifies that ten iterations are required.

Display the output to screen every iteration using plain text, you may assume that the board is 150 cells wide and 90 cells tall. ■

Alternative Rules for The Game of Life

The rules for life could also be phrased in a different manner, that is, give birth if there are two neighbours around an empty cell (B2) and allow an 'alive' cell to survive only if surrounded by 2 or 3 cells (S23). Other rules which are *life-like* exist, for instance *34 Life* (B34/S34), *Life Without Death* (B3/S012345678) and HighLife (B36/S23).



http://en.wikipedia.org/wiki/Life-like_cellular_automaton

Exercise 4.3.2 Write a program that allows the user to input life-like rules e.g. :

```
life B34/S34 lifeboard.lif
```

or

```
life B2/S lifeboard.lif
```

and display generations of boards, beginning with the inital board in the input file. ■

4.4 Life Wars

Inspired by the classic game, *Core Wars*



http://en.wikipedia.org/wiki/Core_War

here we look at a two player version of Conway's Game of Life.

In our game, each of two 'players' submit a Life 1.06 file and cells from these inserted into an empty board. The cells are coded based on which player created them (say '+' and '@'). The game is then run, and the player having most cells left after a fixed number of iterations, over many games, is deemed the winner.

The rules are :

1. The board is 150 cells wide, and 90 cells tall.
2. The board is toroidal; that is, it wraps around from the left edge to the right, and the top to the bottom.
3. Each player can submit a Life 1.06 file that is maximum of 100 lines long, **including** the header line.
4. Since each of the Life 1.06 files describe absolute positions for cells, each player is assigned a random origin for their cells. If there is any collision when attempting to add the cells initially (i.e. both players happen to specify a cell in the same square), then new origins are chosen randomly and the process begun from scratch.
5. The 'standard' B3/S23 rules are used.
6. The colour of cells is never taken into account (i.e. cells are still either 'alive' or 'dead'). The sole exception to this is that when a cell is born, it takes the colour of the majority its neighbours.
7. There are 5000 generations played every game.
8. A running count of how many cells each player has left at the end of each game is kept. The board is cleared and the next game randomly restarted.
9. There are 50 games run in total.
10. The player with the highest number of cells over all 50 games is the winner.

It's easy to extend these rules, of course, to allow for three or more players, but it's unclear for three players what would happen in the majority vote if there was a draw (i.e. a new cell is born based on three cells around it, one belonging to each player. Other rule variants are also possible (e.g. *Highlife* (B36/S23), but once again, the birth rule for 3 or 6 neighbours would cause majority voting issues for two and three players.

Exercise 4.4.1 Write a program that accepts two Life 1.06 files and reports which of them is the winner. The first few games might look like:

```
% /lifewars blinkerpuffer2.lif bigglider.lif

         0    50    12  Player 1
         1   370   141  Player 1
         2   437   281  Player 1
         3   450   602  Player 2
         4   540   623  Player 2
         5   991   629  Player 1
         6  1063   674  Player 1
         7  1211   707  Player 1
         8  1263   735  Player 1
         9  1358   758  Player 1
.
.
.
    Player 1 wins by 7857 cells to 2373 cells
```

■

4.5 Wireworld

Wireworld is a cellular automaton due to Brian Silverman, formed from a 2D grid of cells, designed to simulate digital electronics. Each cell can be in one of four states, either 'empty', 'electron head', 'electron tail' or 'copper' (or 'conductor').

The next generation of the cells follows the rules, where n is the number of electron heads found in the 8-surrounding cells:

- empty $->$ empty
- electron head $->$ electron tail
- electron tail $->$ copper
- copper $->$ electron head if $n == 1$ or $n == 2$
- copper $->$ copper otherwise

See also:



`https://en.wikipedia.org/wiki/Wireworld`



`http://www.heise.ws/fourticklogic.html`

Exercise 4.5.1 Write a program which is run using the `argc` and `argv` parameters to `main`. The usage is as follows :

```
$ wireworld wirefile.txt
```

where `wirefile.txt` is a file specifying the initial state of the board. This file codes empty cells as ' ', heads as 'H', tails as 't' and copper as 'c'. Display the board for 1000 generations using plain text. You may assume that the grid is always 40 cells by 40

■

4.6 Forest Fire

You can create a very simple model of forest fires using a cellular automaton in the form a $2D$ grid of cells. Each cell can be in one of three states; either 'empty', 'tree', or 'fire'. The next generation of cells follows these rules:

- A 'fire' cell will turn into an 'empty' cell.
- A 'tree' that is within the 8-neighbourhood of a 'fire' cell will itself become 'fire'.
- A 'tree' will burn (due to a lightning strike) 1 time in L.
- An 'empty' space will become a 'tree' (spontaneous growth) 1 time in G.

See also:



`https://en.wikipedia.org/wiki/Forest-fire_model`



`https://www.aryan.app/randomstuff/forestfire.html`

You can experiment with different values of L and G, but a useful starting point is $G = 250$ and $L = 10 \times G$.

Exercise 4.6.1 Write a program which creates an empty 2D grid of cells, 80 wide and 30 high. Then, apply the rules above to iterate the simulation, so that the next generation is created from the previous one.

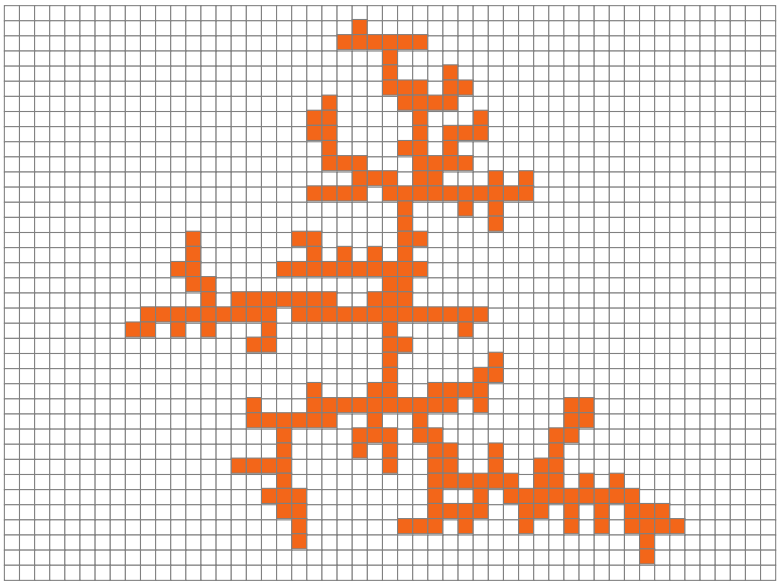
Print out every generation onto the screen, using a space to represent an empty cell, the `@` character for a tree, and `*` for fire cells. Display the board for 1000 generations using plain text. You may assume that the grid is always 80 cells by 30

■

4.7 Diffusion Limited Aggregation

www `https://en.wikipedia.org/wiki/Diffusion-limited_aggregation`

In its simplest form, DLA occurs on a grid of square cells. The cell at the center of the grid is the location of the seed point, a particle stuck at that square. Now pick a square on the perimeter of the grid and place a wandering particle on that square. At each iteration, this particle moves to one of the four adjacent squares, left, right, above, or below. When a wandering particle arrives at one of the four squares adjacent to the seed, it sticks there forming a cluster of two particles, and another edge particle is released. When a moving particle arrives at one of the squares adjacent to the cluster, it sticks there.



Note that the grid is toroidal - this means that if a particle goes off the top of the grid, it reappears at the bottom. Likewise, if it goes off the edge, it will appear on the other side.

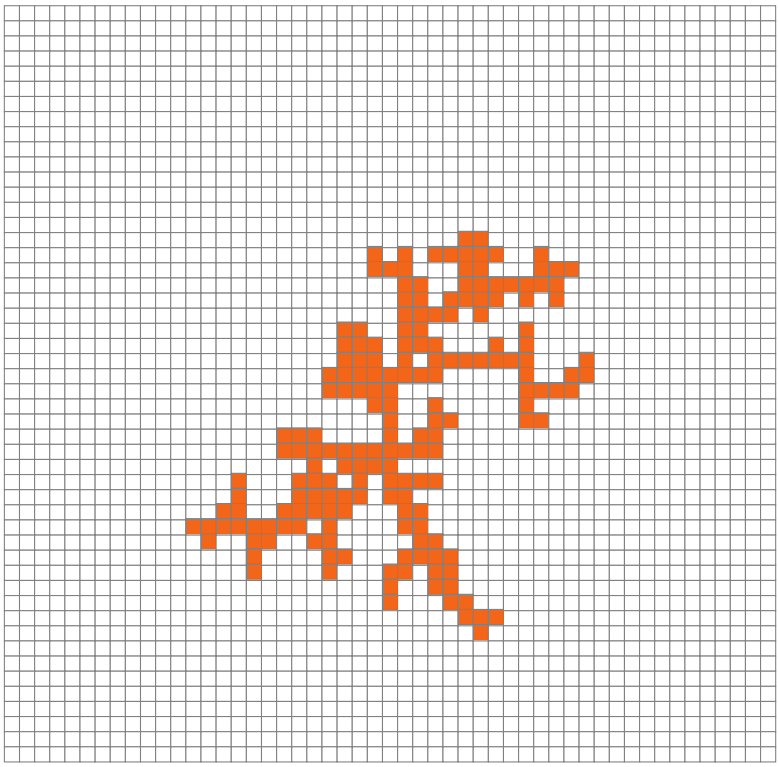
Exercise 4.7.1 Using the algorithm outlined above, using a 50×50 board, output each of 250 iterations (here iteration means when each particle has become 'stuck'). The output should be in plain text. ■

In the basic version of a DLA, a particle 'sticks' to the seed points when it gets adjacent to one. Here we introduce the probability of stickiness, a number between 0 and 1, called p_s. The particle only sticks to a seed point with a probability of p_s; if it doesn't stick it continues wandering. This allows for the patterns that emerge to be more hairy and solid:

www http://paulbourke.net/fractals/dla/

Exercise 4.7.2 Implement this stickiness concept by extending the program written for Exercise 4.7.1, allowing the user to specify a 'stickiness' probability via the command line using argv[1]. Here a value of 1.0 means that the particle will always stick, and 0.5 means that it will stick one time in 2. ■

Here's an example when setting $p_s = 0.25$:



Using different planar walks. e.g. in base-4: http://demonstrations.wolfram.com/UsingIrrationalSqua

4.8 Langton's Ant

www http://en.wikipedia.org/wiki/Langton's_ant

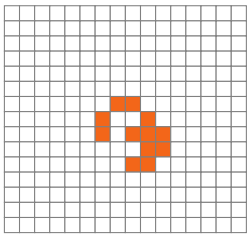
Exercise 4.8.1 Write a program that shows the behaviour of Langton's Ant and displays it on the screen using text. The ant will start in the middle, and each cell will have 2 states : '#' (white) or '.' (black).

At each step, the ant:

- At a white square, turn 90° right, flip the color of the square, move forward one unit
- At a black square, turn 90° left, flip the color of the square, move forward one unit

After each step, display the board, and wait for the user to type a character before repeating. ■

After 30 or so iterations, the board might look something like :



4.9 Minesweeper

The game *Minesweeper*

www `https://en.wikipedia.org/wiki/Minesweeper_(video_game)`

is a logic puzzle game, played on a two-dimensional grid of squares. There are 'mines' hidden in the grid, and other, numbered squares, tell you how many mines there are in that square's (eight-count) Moore neighbourhood.

www `https://en.wikipedia.org/wiki/Moore_neighborhood`

You will know in advance the width and height of the grid, and also the total number of mines in the completed (solved) grid. In our version of the game, we'll be using just two rules to 'solve' the grid by working out the unknown squares.

Rule 1 **(Count the Mines)**: If we've discovered all the mines on the board already, then any unknown cell can simply be numbered with the count of mines in its Moore neighbourhood.

In the following grid, if we somehow know in advance that the total number of mines in the grid is five, and if these have all been found:

0	1	1	?	0
1	3	X	3	1
1	X	X	X	1
1	3	X	3	1
0	1	1	1	0

then the solution to the unknown square must be:

0	1	1	1	0
1	3	X	3	1
1	X	X	X	1
1	3	X	3	1
0	1	1	1	0

Rule 2 **(Unknowns are Mines)**: For a known square having the number n, with u unknown and m known mines in its Moore neighbourhood, if $n = m + u$ and $u > 0$ then all u unknown squares should be made mines.

Applying Rule 2 to this grid:

0	1	1	1	0
1	3	X	3	1
1	X	X	X	1
1	3	?	3	1
0	1	1	1	0

at the middle square on the bottom row (for instance), yields:

0	1	1	1	0
1	3	X	3	1
1	X	X	X	1
1	3	X	3	1
0	1	1	1	0

Repeated application of these rules will allow some (but not all) boards to be solved.

Exercise 4.9.1 Code for this exercise can be found in

www `https://github.com/csnwc/Exercises-In-C`

then navigate into `Code/Week4/Minesweeper`.

Complete the file *ms.c* which, along with my files *ms.h* and *drv.c*, allows the puzzles to be solved.

My file *ms.h* contains the function definitions which you'll have to implement in your **ms.c** file. My file *drv.c* contains the `main()` function to act as a driver to run the code. Your file will contain many other functions as well as those specified, so you'll wish to test them as normal using the `test()` function.

If all of these files are in the same directory, you can compile them using the `Makefile` given. Do not alter or resubmit *ms.h* or *drv.c* - my original versions will be used to compile the *ms.c* file that you create. Only submit *ms.c*

■



5. Files, argc and Graphics

Data for these exercises are available in the github repository *Data*, including lists of English words in *Data/Words*.

5.1 Anagrams

An anagram is a collection of letters that when unscrambled, using all the letters, make a single word. For instance `magrana` can be rearranged to make the word `anagram`.

Exercise 5.1.1 Using a file of valid words, allow the user to enter an anagram, and have the answer(s) printed. For instance :

```
% ./anagram sternaig
angriest
astringe
ganister
gantries
ingrates
rangiest
reasting
stearing
```

An anagram is defined here to be a *different* word, so if a valid word is entered, it itself is not reported, e.g. :

```
% ./anagram pots
tops
stop
opts
spot
post
```

■

Exercise 5.1.2 Using a file of valid words, find all words which are anagrams of each other. Each word should appear in a maximum of one list. Output will look something like :

```
 % ./selfanagram
.
.
7 merits mister miters mitres remits smiter timers
.
.
.
6 noters stoner tenors tensor toners trones
.
.
.
6 opts post pots spot stop tops
.
.
.
6 restrain retrains strainer terrains trainers transire
.
```

■

If you wished to create "interesting" anagrams, rather than simply a random jumble of letters, you could combine together two shorter words which are an anagram of a longer one.

Exercise 5.1.3 Write a program which uses an exhaustive search of all the possible pairs of short words to make the target word to be computed. For instance, a few of the many pairs that can be used to make an anagram of `compiler` are :

```
 % ./teabreak compiler
LiceRomp
LimeCrop
LimpCore
MileCrop
MoreClip
PermCoil
PromLice
RelicMop
```

■

The name `Campbell` comes out as `CalmPleb` which is a bit harsh. Can't **ever** remember being called calm ...

5.2 ANSI Escape Sequences

C has no inherent functionality to allow printing in colour etc. Therefore, some terminals allow certain control codes to be printed to move the cursor, change the foreground and background colour, and so on. Note that only some terminals support these control code - if you're using a different one, garbage may appear on the screen instead.

www `https://en.wikipedia.org/wiki/ANSI_escape_code`

I've created a very simple set of functions which shows a small fraction of this functionality

called `demo_neillsimplescreen.c` which using the functions found in `neillsimplescreen.c`

To build and execute this code, use the `Makefile` provided by typing:

```
$ make demo_neillsimplescreen
gcc demo_neillsimplescreen.c neillsimplescreen.c
-o demo_neillsimplescreen -Wall -Wextra -pedantic
-std=c99 -Wvla -g3 -fsanitize=undefined -fsanitize=address -lm
$ ./demo_neillsimplescreen
```

Exercise 5.2.1 Adapt any of the exercises from Chapter 4, so that output is displayed using the simple functions demonstrated above, rather than plain text. ■

5.3 SDL - Intro

Many programming languages have no inherent graphics capabilities. To get windows to appear on the screen, or to draw lines and shapes, you need to make use of an external library. Here we use SDL[1], a cross-platform library providing the user with (amongst other things) such graphical capabilities.

www `https://www.libsdl.org/`

The use of SDL is, unsurprisingly, non-trivial, so some simple wrapper files have been created (`neillsdl2.c` and `neillsdl2.h`). These give you some simple functions to initialise a window, draw rectangles, wait for the user to press a key etc.

An example program using this functionality is provided in a file `demo_neillsdl2.c`.

This program initialises a window, then sits in a loop, drawing randomly positioned and coloured squares, until the user presses the mouse or a key.

Exercise 5.3.1 Using the `Makefile` provided, compile and run this program. Now adapt it, so that the colour of the boxes displayed are all (random) shades of red.

SDL is already installed on lab machines. At home, if you're using a ubuntu-style linux machine, use: `sudo apt install libsdl2-dev` to install it. ■

5.4 A Simple Spelling Checker

Here, we reads words one at a time from file and carefully place them in the **correct** part of our data structure. This has a complexity of $O(n^2)$.

For this purpose, a list of valid words (unsorted) is available from `github : Data/Words`.

Exercise 5.4.1 Write a program which, based on a fixed-size **array** of strings, reads the words in one at a time, inserting them into the **correct** part of the array so that the words are alphabetically sorted. The name of the file should be passed as `argv[1]`, and you can assume the array is large enough to hold all words. How long does it take to build the list ? ■

Exercise 5.4.2 Now extend Exercise 5.4.1 so that when the user is prompted for a word, they are told whether this word is present in the array or not. Use a *Linear Search* for this: start at the beginning of the list and work your way through it one word at a time. ■

[1] actually, we are using the most recent version SDL2, which is installed on all the lab machines

Exercise 5.4.3 Extend Exercise 5.4.2 using a *Binary Search* for this. ■

Exercise 5.4.4 Write a program which, based on a linked list data structure, reads the words in one at a time, inserting them into the **correct** part of the list so that the words are alphabetically sorted. The name of the file should be passed as `argv[1]`. How long does it take to build the list ?
How long does it take to do a linear search ? ■

5.5 Crush It!

Match-3 tile games have become one of the world's most popular games.

www `https://en.wikipedia.org/wiki/Tile-matching_video_game`

Such games use a rectangular grid (board) containing many tiles, of many different types (often colour, but here we will use letters). Where there are 3 or more tiles of the same type in a line (horizontally or vertically), they are removed. Once all removals that are possible have occurred, the tiles above fall down to fill in the gaps. In our version, a large number of tiles are available above (and hidden from the player) which cannot be matched until they have fallen down into the playing area.

In our version of the game:

- Matchable tiles are in the range $(A \ldots Z)$, although it's is common for only a small number (e.g. $A \ldots D$) to be used.
- The width of the board is always five tiles.
- The 'playing' height of the board is six. Other tiles can be above this, but won't be matched until they have dropped down into one of the bottom six rows.
- The maximum number of rows the board ever needs to hold is 20.
- Matching (that is finding a horizonal or vertical line of the same tile) is done in 'parallel' - if a tile is shared between two matches (e.g. the middle tile in a 3×3 '+' pattern) both of these matches are removed.
- Given the limited size of board in which matches can be made, the longest match that can be made is of five tiles horizontally (the width of the board) and six tiles vertically the 'playing' height of the board.

An example of this is shown here:

·	·	·	·	·
B	B	B	D	B
C	D	A	A	C
D	A	A	B	D
A	A	B	C	A
A	B	C	D	A
B	C	A	A	A

·	·	·	·	·
·	·	·	D	B
C	D	A	A	C
D	A	A	B	D
A	A	B	C	·
A	B	C	D	·
B	C	·	·	·

·	·	·	·	·
·	·	·	·	·
C	D	·	D	·
D	A	A	A	·
A	A	A	B	B
A	B	B	C	C
B	C	C	D	D

(Left) Game board in it's initial state. Orange squares show were matches can be made. (Middle) Three matches are made - one for the 3 horizonal 'A' tiles, one for the three vertical 'A' tiles and one for the three horizontal 'B' tiles. (Right) Game board final state after tiles are dropped down. Not there are now more matches that can be made.

Exercise 5.5.1 Here we will write some (but not all) of the functionality necessary for a match-3 tile game. Skeleton code may be found in :

www `https://github.com/csnwc/Exercises-In-C`

then navigate into `Code/Week5/CrushIt`.

Complete the files **crushit.c** and **mydefs.h** which, along with my files *crushit.h* and *driver.c*, implements some important functionality necessary for a game of this type.
My file *crushit.h* contains the function definitions which you'll have to implement in your **crushit.c** file. My file *driver.c* contains the `main()` function to act as a driver to run the code. Your file will contain many other functions as well as those specified, so you'll wish to test them as normal using a `test()` function.
If all of these files are in the same directory, you can compile them using the `Makefile` given. The functions you need to complete include:

`initialise()` - this takes a pointer to the board state, and a string. The string can be a filename, but if this filename can't be opened, it is assumed to be a list of tiles to fill the board with, from the top down. Such a string must contain complete rows of tiles, with no partial rows.

`match()` - this takes a pointer to the board state, and removes all matches of 3 or more tiles in a vertical or horizontal line. Removed tiles are replaces with the '.' (empty tile) character.

`dropblocks()` - this takes a pointer to the board state, and makes any blocks that are above an empty tile fall down until all holes are filled in if it is possible to do so.

`tostring()` - this takes a pointer to the board state, and a string and copies whole rows of the board into the string from the top downwards. The whole board isn't copied since most of the characters at the top are unused (hole) tiles. Therefore, we begin copying at the first row on which a non-hole tile appears.

Hints:

- Do not begin by writing the file handling functionality - this cannot be tested, so use the string initialising option instead.
- To begin with use your own, simpler driver file - mine makes sense once everything is working, but may seem complex to begin with.
- Your `crushit.c` file should contain many other sub-functions which are used by the major ones specified. You can put anything in this file, provided it still compiles as specified.
- Do not alter or resubmit *crushit.h*, *Makefile* or *driver.c* - my original versions (or even slightly different ones) will be used to compile the *crushit.c* file that you adapt.

■

5.6 Tents & Trees

Tents is a logic puzzle in which you work out where to place tents next to trees in a field:

The rules are explained at:

www `https://www.puzzle-tents.com`

- Pair each tree with a tent adjacent horizontally or vertically (4-connected). This should be a 1 to 1 relation.
- Tents never touch each other even diagonally (8-connected)

- The clues outside the grid indicate the number of tents on that row/column.

The game consists of a **square** board representing a field where every tile may be a 'tree', a 'tent', 'grass' or unknown. Trees are represented by the character '*', tents using '+', grass with '.' and unknowns as ' '. An example of a completed puzzle is shown here:

	1	2	0	1	2	1
2	*					*
0					*	
1						
1			*			*
1						
2		*	*			

	1	2	0	1	2	1
2	*	+	.	.	+	*
0	.	.	.	.	*	.
1	.	.	.	.	+	.
1	.	+	*	.	.	*
1	.	.	.	.	.	+
2	+	*	*	+	.	.

(Left) Puzzle as defined - only the trees are given to begin with, all other tiles are unknown. There are 7 tents to be found, the number of them in total per row and column is also shown. (Right) The solved puzzle after applying some simple logic.

Exercise 5.6.1 We will implement five rules which will allow **some**, but **not** all, puzzles to be solved:

1. If a row or column already has the correct number of tents, all unknowns → grass.
2. Any unknown cell not 4-connected to a tree → grass.
3. If the number of unknown cells on a line equals the number of tents still left to be found, all unknowns → tents.
4. Any unknown cell 8-connected to a tent → grass.
5. For any tree and its 4-connected cells, if there is exactly one unknown and zero tents, then that unknown → tent.

Some skeleton code may be found in :

www `https://github.com/csnwc/Exercises-In-C`

then navigate into `Code/Week5/TentsTrees`.

You may assume that:

- The size of the board is always square. Its exact size is `#defined` (via the Makefile).
- You will never be expected to solve a board any larger than 9*x*9.

Complete the file **tt.c** which, along with my files *driver.c* and *tentstrees.h*, implements some important functionality necessary for a game of this type.

My file *tentstrees.h* contains the function definitions which you'll have to implement in your **tt.c** file. My file *driver.c* contains the `main()` function to act as a driver to run the code. Your file will contain many other functions as well as those specified, so you'll wish to test them as normal using a `test()` function.

If all of these files are in the same directory, you can compile them using the `Makefile` given. The functions you need to complete include:

`inputboard()` - this takes a string corresponding to the initial state of the board (including row seperator characters '|', two strings containing the number of tents in each column/row, and a pointer to the board state to be filled in. It returns a Boolean, true if all is OK, false if the number of characters in the input is incorrect, or if the total number of tents in the rows is different to that in the columns, or if incorrect characters have been used - see

the assertions for more examples of failure.

`solve()` - this takes a pointer to the board state, and returns a Boolean, depending on whether the five rules, applied repeatedly, can solve this board or not.

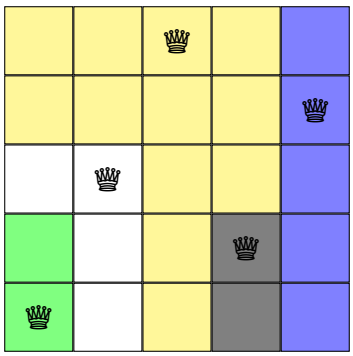
`board2str()` - this takes a copy of the board state, and copies this board into the string, in the same format as required for `inputboard`. It returns a Boolean, with a NULL string causing a failure.

Hints:

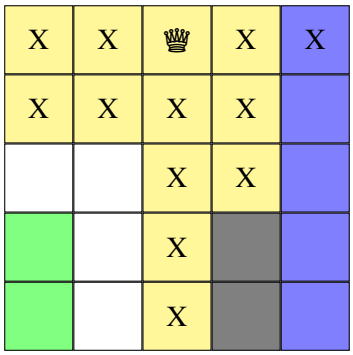
- To begin with use your own, simpler driver file - mine makes sense once everything is working, but may seem complex to begin with.
- Your `tt.c` file should contain many other sub-functions which are used by the major ones specified. You can put anything in this file, provided it still compiles as specified.
- Do not alter or resubmit *driver.c*, *Makefile* or *tentstrees.h* - my original versions (and variants) will be used to compile the *tt.c* file that you write.
- If you have questions such as *"What happens if a NULL pointer is passed to solve()"* - I'm happy for you to make your own judgement, provided that your code does something graceful i.e. cannot be crashed.
- Your submitted program will not print anything to screen.
- This exercise is about solving this logic puzzle exactly as specified. For now, I'm not interested in you finding other, better rules etc. It's a programming assignment above all else.

5.7 Crowns

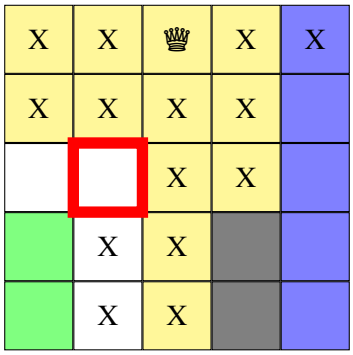
Crowns (sometimes called Queens or Star Battle) is a logic puzzle in which you place crowns on a board so that there is only one Crown per row, column, and coloured region. Each row, column, and coloured region must contain exactly one Crown symbol (Queen). Crown symbols cannot be placed in adjacent cells, including the eight cells immediately neighbouring the crown.



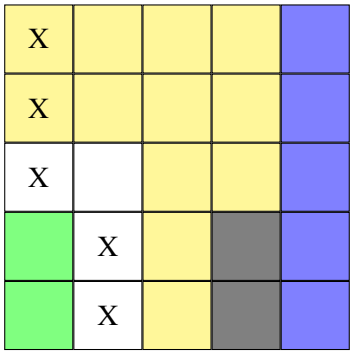
During the game, players mark cells that cannot be crowns with crosses. Every cell needs two values - its region colour (or symbol) and its current state (unknown, crossed, crown). When you place a crown, you can cross out all other cells on that row, column, in the eight-neighbourhood and in cells having the same colour. Here, a crown has been placed in the yellow region, so many cells have been crossed out:



There are other some simple rules you could think of to help you find a solution. These include the **singleton** rule; any row, column or colour region having no current crown, and only one unknown, must have a crown in that cell. Here, one column has only one unknown, and so we know we should place a crown in the highlighted cell:



Another rule is that if, by placing a crown in a cell (and therefore all the crosses generated on the same row, column, region and eight-neighbours), you remove all possible solutions for another region, then that cell cannot be a crown and should be crossed. Here, some cells have been crossed because if they were crowns they would lead to no valid placements in the green region (this rule would also cross other cells, including those around the grey region).



A computer player (rather than human) could also examine all the possible permuations of possible crown positions, and simply choose the one that works correctly.

Exercise 5.7.1 Here we will implement a few of the rules which might be required for a full version of the game, including some that would help provide hints to a player, and one that always guarantees a solution if one exists.
Some skeleton code may be found in :

www `https://github.com/csnwc/Exercises-In-C`

then navigate into `Code/Week5/Crowns`.

Complete the file **crowns.c** which, along with my files *driver.c* and *crowns.h*, implements some important functionality necessary for a game of this type.
My file *crowns.h* contains the function definitions which you'll have to implement in your **crowns.c** file. My file *driver.c* contains the `main()` function to act as a driver to run the code. Your file will contain many other functions as well as those specified, so you'll wish to test them as normal using a `test()` function.
If all of these files are in the same directory, you can compile them using the `Makefile` given. The functions you need to complete include:

```
bool str2board(board* b, char* s);
```

s is a string defining the board, two characters per cell. The first is the region character, and the second is how this cell is marked either ' ' for unknown, 'X' for crossed and '*' for crown. The character 'I' is used to separate rows. Returns `true` if the string can be parsed, `false` otherwise. Don't check whether the board is solvable or not - if the user wants to input an unsolvable 3×3 board, let them.

```
bool board2str(char* str, const board* b);
```

Given a board b, write the 'stringified' version of it into str.

```
int count_unknowns(board b);
```

For a given board count the number of unknowns (i.e. cells which are not marked as crossed or crowns).

```
bool rule1(board* b);
```

Takes a board b and adds all the crosses induced by crowns placed in it. Returns `true` if any changes are made, `false` otherwise.

```
bool rule2(board* b);
```

Takes a board b and adds any crowns due to any row, column, or region having only one unknown remaining. Repeatedly applies this rule, and then rule1 until no more changes are possible. Returns `true` if any changes have been made at all, or `false` otherwise.

```
bool rule3(board* b);
```

Takes a board `b` and adds crosses to any cells that, if they had a crown placed there instead, would cause another region to have no cells available for crowns. Repeatedly applies this until no more changes are possible. Returns `true` if any changes have been made at all, or `false` otherwise.

```
bool lut(int sz, int n, char str[]);
```

By compiling with `lut.o` you are provided with the function `lut()`, which fills the string `str` corresponding to the nth legal solution to a board of size `sz` (ignoring region colours). You can iterate over all these solutions, until `lut()` returns `false` to show that there are no more solutions. This forms the basis for `rule5()`. If `sz` is less than 4, or greater than the maximum board size, `false` is returned. Returns `false` when `n` becomes too large for a particular board size; e.g. a board of size 4 has only 2 possible solutions.

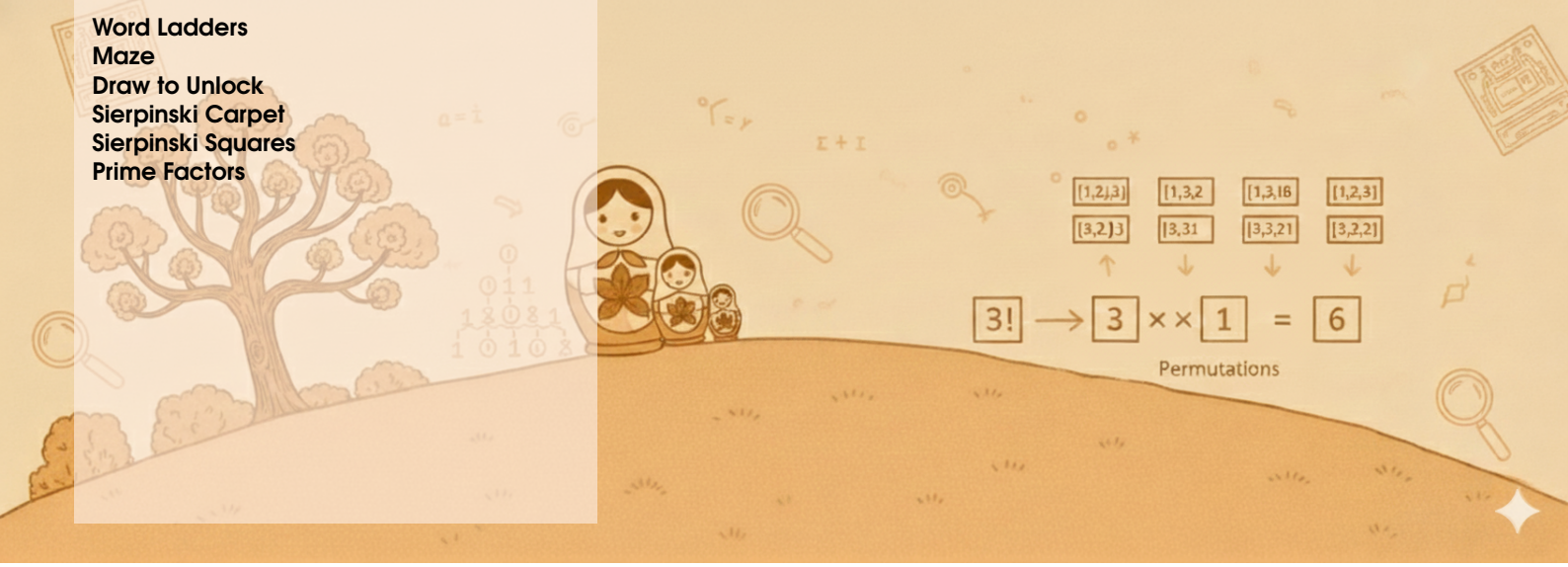
```
bool rule5(board* b);
```

This is the brute-force rule for finding the (hopefully, unique) solution. Takes a board `b` and, using all possible solutions available by iterating `lut()` finds the correct solution and completes the board. Returns `true` if a solution is possible, or `false` otherwise.
Hints:

- The size of the board is always square. The maximum exact size is `#defined` in `crowns.h`.
- The 'colour' of each cell/region is represented by characters chosen on input e.g. ('w', 'r', 'g'). Each cell also has a character corresponding to its current state (crown, cross, unknown).
- To begin with, use your own, simpler driver file - mine makes sense once everything is working, but may seem complex to begin with.
- Your `crowns.c` file should contain many other sub-functions which are used by the major ones specified. You can put anything in this file, provided it still compiles as specified.
- Do not alter or resubmit *driver.c*, *Makefile* or *crowns.h* - my versions (and possibly, other, similar versions) will be used to compile the *crowns.c* file that you write.
- If you have questions such as *"What happens if a NULL pointer is passed to rule1()"* - I'm happy for you to make your own judgement, provided that your code does something graceful i.e. cannot be crashed.
- Your submitted `crowns.c` will not print anything to screen.
- This exercise is about solving this logic puzzle exactly as specified. For now, I'm not interested in you finding other, better rules etc. It's a programming assignment above all else.

■

Word Ladders
Maze
Draw to Unlock
Sierpinski Carpet
Sierpinski Squares
Prime Factors



7. Recursion

7.1 Word Ladders

In this game, made famous by the author Lewis Caroll, and investigated by many Computer Scientists including Donald Knuth, you find missing words to complete a sequence. For instance, you might be asked how go from "WILD" to "TAME" by only changing one character at a time:

W I L D
W I L E
T I L E
T A L E
T A M E

In a heavily constrained version of this game we make some simplifying assumptions:

- Words are always four letters long.
- We only seek ladders of five words in total.
- Only one letter is changed at a time.
- A letter is only changed from its initial state, to its target state. This is important, since if you decide to change the second letter then you will always know what it's changing from, to what it's changing to.

So, in the example above, it is enough to give the first word, the last word, and the position of the character which changed on each line. On line one, the fourth letter 'D' was changed to an 'E', on the next line the first character 'W' was changed to a 'T' and so on. The whole ladder can be defined by "WILD", "TAME" and the sequence $4, 1, 2, 3$.

W I L D
W I L E
T I L E
T A L E
T A M E

Since each letter changes exactly once, the order in which this happens is a *permutation* of the numbers $1, 2, 3, 4$, which we have looked at elsewhere.

Exercise 7.1.1 For the constrained version of the game, given a file of valid four letter words, write a program which when given two words on the command line (`argv[1]` and `argv[2]`) outputs the correct solution, if available. Use an exhaustive search over the 24 permutations until one leads to no invalid words being required. Make sure your program works, with amongst others, the following:

C O L D	P O K E	C U B E
C O R D	P O L E	C U B S
C A R D	P O L L	T U B S
W A R D	M O L L	T U N S
W A R M	M A L L	T O N S

■

Exercise 7.1.2 Adapt the program above so that if the first and last words share a letter (the edit distance is less than 4), you can find the word ladder required, as in:

W A S P
W A S H
W I S H
F I S H

■

For the "full" version of Wordladder, you make no assumptions about the number of words that are needed to make the ladder, although we do assume that all the words in the ladder are the same size.

Exercise 7.1.3 To achieve this, you could make a list of all the words, and for all words an `edit distance` of 1 away from the initial word, mark these and store their 'parent'. Now, go through this list, and for all words marked, find words which are distance 1 from these, and hence distance 2 from the initial word. Mark these and retain their parent. Be careful you don't use words already marked. If the word ladder is possible, you'll eventually find the solution, and via the record of the parents, have the correct route. This is shown for the word ladder CAT to DOG in Figure 7.1 using a very small subset of the possible three letter words. ■

7.2 Maze

New This exercise is new for 2023

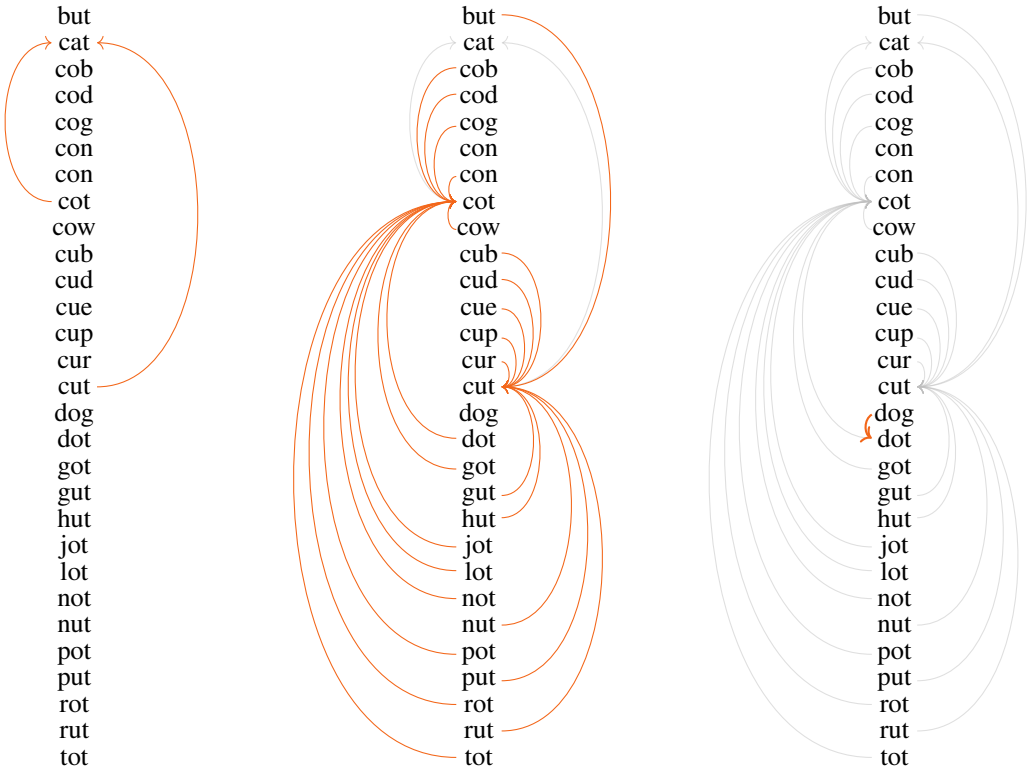
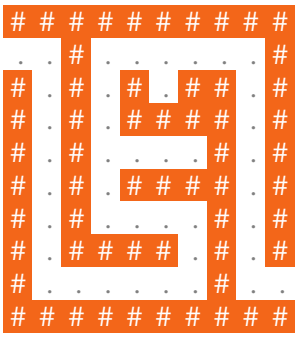


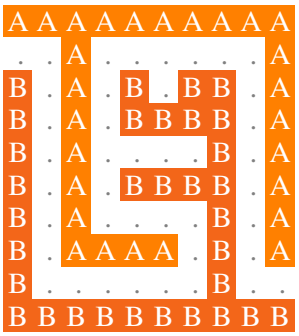
Figure 7.1: Word Ladder from CAT to DOG. (Left) Words which are distance one from CAT are COT and CUT. (Middle) Words which are distance one from CUT and COT, which includes amongst others, DOT. (Right) DOG is distance one from DOT. We now have our route, via the pointers, back to CAT.



Escaping from a maze can be done in several ways (ink-blotting, righthand-on-wall etc.) but here we look at recursion.

Exercise 7.2.1 Write a program to read in a maze typed by a user via the filename passed to `argv[1]`. You can assume the maze will be no larger than 20×20, walls are designated by a `#` and the rest are spaces. The entrance can be assumed to be the gap in the wall closest to (but not necessarily exactly at) the top lefthand corner. The sizes of the maze are given on the first line of the file (width,height). Write a program that finds the route through a maze, read from this file, and prints out the solution (if one exists) using full stops. If the program succeeds it should exit with a status of `0`, or if no route exists it should exit with a status of `1`. ■

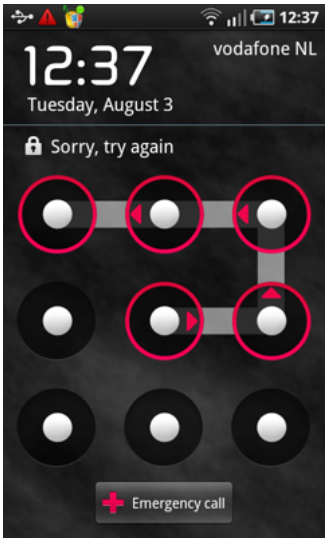
It becomes obvious that the walls of every maze (having one unique solution) must consist of two separate sections :



Exercise 7.2.2 Write a program to read in a maze in the same manner as in Exercise 7.2.1, and then display the two sections using the characters `A` and `B`. ■

7.3 Draw to Unlock

Rather than remembering passwords or passcodes, many mobile devices now allow the user to draw a pattern on the screen to unlock them.



Here we will explore how many unique patterns are available when drawing such patterns to connect "dots", such as shown in the figure. We assume that people put their finger on one "dot" and then only ever move one position left, right, up or down (but never diagonally) at a time. You are not allowed to return to a "dot" once it has been visited once. If we number the first position in our path as 1, the second as 2 and so on, then beginning in the top left-hand corner, some of the possible patterns of 9 moves are :

```
1 2 3      1 2 3      1 2 3
6 5 4      8 9 4      8 7 4
7 8 9      7 6 5      9 6 5
```

Exercise 7.3.1 Write a program that computes and outputs all the valid paths. Use **recursion** to achieve this.

- How many different patterns of length 9 are available on a 3×3 grid, if the user begins in the top left corner ?
- How many different patterns of length 9 are available on a 3×3 grid, if the user begins in the middle left ?
- How many different patterns of length 7 are available on a 3×3 grid, if the user begins in the top left corner ?
- How many different patterns of length 25 are available on a 5×5 grid, if the user begins in the top left corner ?

■

7.4 Sierpinski Carpet

www `en.wikipedia.org/wiki/Sierpinski_carpet`

The square is cut into 9 congruent subsquares in a 3-by-3 grid, and the central subsquare is removed. The same procedure is then applied recursively to the remaining 8 subsquares, ad infinitum.

www `http://www.evilmadscientist.com/2008/sierpinski-cookies/`

Exercise 7.4.1 Write a program that, in plain text, produces a Sierpinski Carpet. ■

Exercise 7.4.2 Write a program that, using `neillsimplescreen`, produces a Sierpinski Carpet. ■

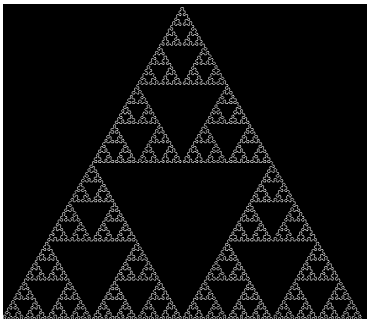
Exercise 7.4.3 Write a program that, using SDL, produces a Sierpinski Carpet. ■

7.5 Sierpinski Squares

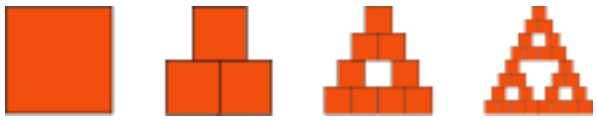
See also :

www `en.wikipedia.org/wiki/Sierpinski_triangle`

The Sierpinski triangle has the overall shape of an equilateral triangle, recursively subdivded into four smaller triangles :



However, we can approximate it by recursively drawing a square as three smaller squares, as show below :



The recursion should terminate when the squares are too small to draw with any more detail (e.g. one pixel, or one character in size).

Exercise 7.5.1 Write a program that, in plain text, produces a Sierpinski Triangle. ■

Exercise 7.5.2 Write a program that, using `neillsimplescreen`, produces a Sierpinski Triangle. ■

Exercise 7.5.3 Write a program that, using SDL, produces a Sierpinski Triangle. ■

7.6 Prime Factors

www `en.wikipedia.org/wiki/Prime_factor`

It is well known that any positive integer has a single *unique* prime factorization, e.g.:

$210 = 7 \times 5 \times 3 \times 2$ (the numbers $7, 5, 3$ and 2 are all prime).
$117 = 13 \times 3 \times 3$ (the numbers 13 and 3 are all prime).
197 is prime, so has only itself (and 1, which we ignore) as a factor.

Exercise 7.6.1 Write a program that, for any given positive integer input using `argv[1]`, lists the prime factors, e.g.:

```
[campbell@icy]% ./primefacts 210
7 5 3 2

[campbell@icy]% ./primefacts 117
13 3 3
```

■

Exercise 7.6.2 To make the output of the above program briefer, many prefer to show the factors expressed by their power, as in :

$$768 = 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 3$$

could be better expressed as :

$$768 = 2^8 \times 3$$

Write a program to show the factorisation of a number in this more compact style :

```
 % ./primefactors 27000
27000 = 1 x 2^3 x 3^3 x 5^3
 % ./primefactors 31
31 = 1 x 31
 % ./primefactors 38654705664
38654705664 = 1 x 2^32 x 3^2
```

■

Exercise 7.6.3



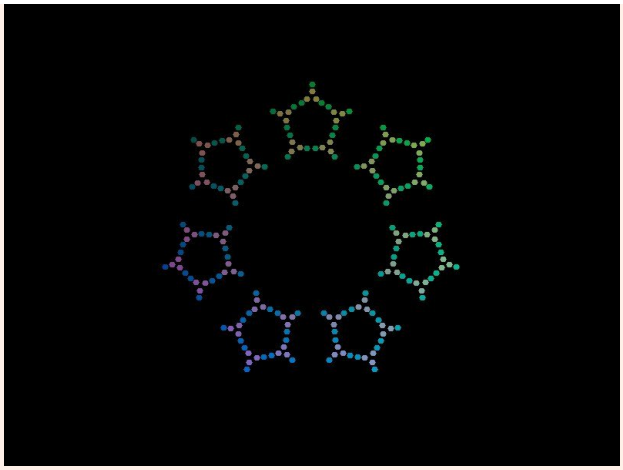
Experience shows that the next exercise is trickier than most - you have been warned!

For a beautiful visualisation of prime factors, see:



`www.datapointed.net/visualizations/math/factorization/animated-diagrams`

Adapt the program above to output a pattern similar to the animated display above, using SDL, but only for a single number given via `argv[1]`.



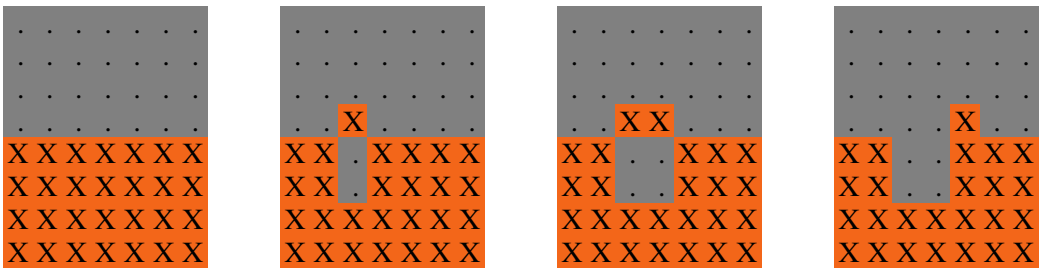
Conway's Soldiers
The 8-Tile Puzzle
Happy Bookcases
Roller-Board
Car Park
The N-Queens Puzzle
Match Drop
NumMatch



8. Searching Boards

8.1 Conway's Soldiers

The one player game, *Conway's Soldiers* (sometimes known as *Solitaire Army*), is similar to peg solitaire. For this exercise, Conway's board is a 7 (width) × 8 (height) board with tiles on it. The lower half of the board is entirely filled with tiles (pegs), and the upper half is completely empty. A tile can move by jumping another tile, either horizontally or vertically (but never diagonally) onto an empty square. The jumped tile is then removed from the board. A few possible moves are shown below:



The user enters the location of an empty square they'd like to get a tile into, and the program demonstrates the moves that enables the tile to reach there (or warns them it's impossible). To do this you will use a list of boards. The initial board is put into this list. Each board in the list is, in turn, read from the list and all possible moves from that board added into the list. The next board is taken, and all its resulting boards are added, and so on.

Each structure in the list will contain (amongst other things) a board and a record of its parent board, i.e. the board that it was created from.

Exercise 8.1.1 Write a program that:

- Inputs a target location for a tile to reach (x in argv[1], y in argv[2]).
- Demonstrates the correct solution (reverse order is fine) using plain text.

■

Use the algorithm described above and not anything else.

8.2 The 8-Tile Puzzle

The Chinese 8-Tile Puzzle is a 3×3 board, with 8 numbered tiles in it, and a hole into which neighbouring tiles can move: 

After the next move the board could look like: 

or 

The problem generally involves the board starting in a random state, and the user returning the board to the 'ordered' "12345678" state.

In this problem, a solution could be found in many different ways; the solution could be recursive, or you could implement a queue to perform a breadth-first search, or something more complex allowing a depth-first search to measure 'how close' (in some sense) it is to the correct solution.

Exercise 8.2.1 Read in a board using `argv[1]`, e.g.:

```
$ 8tile "513276 48"
```

To do this you will use a list of boards. The initial board is put into this list. Each board in the list is, in turn, read from the list and all possible moves from that board added into the list. The next board is taken, and all its resulting boards are added, and so on. This is, essentially, a queue.

However, one problem with is that repeated boards may be put into the queue and 'cycles' occur. This soon creates an explosively large number of boards (several million). You can solve this by only adding a board into the queue if an identical one does not already exist in the queue. A linear search is acceptable for this task of identifying duplicates. Each structure in the queue will contain (amongst other things) a board and a record of its parent board, i.e. the board that it was created from.

Be advised that a solution requiring as 'few' as 20 moves may take 10's of minutes to compute. If the search is successful, display the solution to the screen using plain-text.

Use the method described above and only this one. Use a static data structure to achieve this (arrays) and **not** a dynamic method such as linked-lists; a (large) $1D$ array of structures is acceptable. Because this array needs to be so large, it's best to declare it in `main()` using something like:

```
static boards[NUMBOARDS];
```

■

Exercise 8.2.2 Repeat Exercise 8.2.1, but use SDL for the output rather than plain-text. ■

Exercise 8.2.3 Repeat Exercise 8.2.1, but using a dynamic (linked-list), so that you never have to make any assumptions about the maximum numbers of boards stored. ■

Exercise 8.2.4 Repeat Exercise 8.2.3, but using a 5×5 board instead. ■

8.3 Happy Bookcases

In a quiet part of our building, there are some rather strange bookcases. They are (like most bookcases) generally happy, but they become unhappy when their books are not arranged

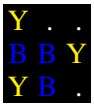
correctly (which, even in a School of Computer Science, is somewhat unusual). After years of dedicated research, a team of scientists led by Simon Lock and Sion Hannuna came to understand the trick to making the bookcases happy again. It turned out that a bookcase is only happy if :

- Each shelf only has books of one colour (or is empty).
- All books of the same colour are on the same shelf.
- The only books that exists are black(K), red(R), green(G), yellow(Y), blue(B), magenta(M), cyan(C) or white(W).

However, to make things worse, there are some complex rules about how books may be re-arranged :

1. You can only move one book at a time.
2. The only book that can move is the rightmost one from each shelf.
3. The book must move to become the rightmost book on its new shelf.
4. You can't put more books on a shelf than its maximum size.

So, for instance, the bookcase below has three shelves, each of which can fit three books; the first contains only one book, the second has three books, and the third shelf has two books on it.



By following the rules, highly-trained librarians can rearrange the books to make the bookcase happy again. One such way of re-arranging the books correctly is shown below :



Here's another example of an unhappy bookcase, and how to rearrange the books to make it happy :



Exercise 8.3.1 Write a program that reads in a bookcase definition file (specified on the command line), and shows the 'moves' to make the bookcase happy. Such a file looks something like :

```
4 3 7
RG.
GR.
CY.
YC.
```

The first line has two or three numbers on it; the height of the bookcase (number of shelves), the width (maximum books per shelf) and an **optional** hint as to the minimum number of bookcases involved when 'solving' this bookcase. (This number is meaningless for a bookcase that cannot be made happy.) The number includes the original bookcase, and the final 'happy' one in the count. For the bookcase shown in this file, one possible solution is :



In the file, an empty space is defined by a full-stop character. You may assume that the

maximum height and width of a bookcase is 9.
The brute-force algorithm for searching over all moves to make the bookcase happy goes like this :

1. You will use a list of bookcases (here list could either be an array, or a linked list).
2. The initial bookcase is put into the front of this list.
3. Take a bookcase from the **front** of the list.
4. For this (parent) bookcase, find the resulting (child) bookcases which can be created from all the valid possible single book moves. Put each of these bookcases into the **end** of the list. There may be as many as $height \times (height - 1)$ of these. If you have found a happy bookcase, stop. Else, go to 3.

To help with printing out the correct moves when a solution has been found, each structure in the list will need to contain (amongst other things) a bookcase and a record of its parent bookcase, i.e. the bookcase that it was created from. For an array, this could simply be which element of the array was the parent, or for a linked list, this will be a pointer.
The program reads the name of the bookcase definition file from `argv[1]`. If it finds a successful way to make the bookcase happy, it prints out the number of bookcases that would be printed in the solution and **nothing else**, or else exactly the phrase 'No Solution?" if none can be found :

```
$ ./bookcase rrggccyy-437.bc
7
$ ./bookcase rrrr-22.bc
No Solution?
$ ./bookcase ccbb-23.bc
1
```

If the 'verbose' flag is used (argv[2]), your program will additionally print out the solution (reverse order is fine) :

```
$ ./bookcase ccbb-23.bc verbose
1

CC.
BB.

$ ./bookcase rgbrmrykwrrr-3521.bc verbose
No Solution?

$ ./bookcase yby-222.bc verbose
2

Y.
BY

YY
B.
```

Your program :

- **Must** use the algorithm detailed above (which is similar to a queue and therefore a breadth-first search). Other search algorithms are possible (e.g. best-first, guided, recursive etc.) but the quality of coding is being assessed, not the quality of the algorithm used!

- **Should** check for invalid bookcase definition files, and report in a graceful way if there is a problem, aborting with `exit(EXIT_FAILURE)` if so.
- **May** display the bookcases in colour if you wish - if so use `neillsimplescreen` to do so.
- **Should not** print anything else out to screen after successfully completing the search, except that which is shown above. Automated checking may be used, and therefore the output must be precise.
- **Should** call the function `test()` to perform any assertion testing etc.

Extension

Basic assignment = 90%. Extension = 10%.
If you'd like to try an extension, make sure to submit *extension.c* and a brief description in a *extension.txt* file. This could involve a faster search technique, better graphical display, user input or something else of your choosing. The extension will be marked in the same way as the main assignment.

■

8.4 Roller-Board

The puzzle *Roller-Board* consists of a $2D$ rectangular grid of cells, each of which is labelled either '0' or '1':

```
0 0 0 0 0
0 1 0 1 0
0 0 1 0 0
0 1 0 1 0
0 0 0 0 0
```

The challenge is to roll one row or column at a time, so that the board is returned to its 'correct' state:

```
1 1 1 1 1
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
```

having all $1s$ on the top row, and every other cell being a 0. Each 'move' can be either:

- Roll a column one place up - i.e. the cells in this column all move up one, and the cell at the top 'rolls around' and reappears at the bottom of this column.
- Roll a column one place down - i.e. the cells in this column all move down one, and the cell at the bottom 'rolls around' and reappears at the top of this column.
- Roll a row one place left - i.e. the cells in this row all move left one, and the cell on the left 'rolls around' and reappears on the right of this row.
- Roll a row one place right - i.e. the cells in this row all right one, and the cell on the right 'rolls around' and reappears on the left of this row.

To solve a 4×4 board:

```
1 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1
```

the best solution might be:

Roll column 1 up :

1 1 0 0
0 0 0 0
0 0 1 0
0 0 0 1

Roll column 2 down :

1 1 0 0
0 0 0 0
0 0 0 0
0 0 1 1

Roll column 2 down :

1 1 1 0
0 0 0 0
0 0 0 0
0 0 0 1

Roll column 3 down :

1 1 1 1
0 0 0 0
0 0 0 0
0 0 0 0

Here's another example of a board:

0 0 0 0
1 1 0 0
1 0 0 0
0 1 0 0
0 0 0 0

and how to solve it:



Exercise 8.4.1 Write a program that reads in a roller-board file (specified on the command line), and shows the 'moves' to solve it. Such a file looks something like :

```
5 4
0000
1100
1000
0100
0000
```

The first line has two numbers; the height of the board (number of rows) and then the width (number of columns).
In the remainder of the file, the number of 1s must be equal to the width of the board, and only the characters '0' and '1' are valid.You may assume that the maximum height and width of a board is 6.
The brute-force algorithm for searching over all moves for a solution goes like this :

1. You will use an `alloc()`'d array (list) of boards.
2. Put the initial board into the front of this list, `f=0`.
3. Consider the board at the **front** of the list (index `f`).
4. For this (parent) board, find the resulting (child) boards which can be created from all the possible $(rows + columns) \times 2$ rolls. For each of these child boards:
   - If this board is unique (i.e. it has not been seen before in the list), add it to the end of the list.

- If it has been seen before (a duplicate) ignore it.
- If it is the 'final' board, stop and print the solution.

5. Add one of f. If there are more boards in the list, go to step 3.

To help with printing out the correct moves, when a solution has been found, each structure in the list will need to contain (amongst other things) a board and a record of its parent board, i.e. the board that it was created from. Since you're using an array, this could simply be which element of the array was the parent.

The program reads the name of the board definition file from the command line. If it finds a successful solution, it prints out the number of boards that would be printed in the solution and **nothing else**, or else exactly the phrase 'No Solution?" if none can be found (as might be the case if you simply run out of memory) :

```
$ ./rollerboard 4x4diag.rbd
4 moves
$ ./rollerboard 5x5lhs.rbd
8 moves
```

If the 'verbose' flag is used, your program will print out the solution in the correct order :

```
$ ./rollerboard -v 4x4lr.rbd
0:
0000
1001
1001
0000

1:
1000
1001
0001
0000

2:
0100
1001
0001
0000

3:
1100
0001
0001
0000

4:
1101
0001
0000
0000

5:
1110
0001
0000
```

```
0000

6:
1111
0000
0000
0000

$ ./rollerboard -v 3x3crn.rbd
0:
000
001
011

1:
000
100
011

2:
100
000
011

3:
110
000
001

4:
111
000
000
```

Your program :

- **Must** use the algorithm detailed above (which is similar to a queue and therefore a breadth-first search). Do not use the other algorithms possible (e.g. best-first, guided, recursive etc.); the quality of your coding is being assessed against others taking the same approach.
- **Must not** use dynamic arrays or linked lists. Since boards cannot be any larger than 6×6, you can create boards of this size, and only use a sub-part of them if the board required is smaller. The list of boards can be a fixed (large) size.
- **Should** check for invalid board definition files, and report in a graceful way if there is a problem, aborting with `exit(EXIT_FAILURE)` if so.
- **Should not** print anything else out to screen after successfully completing the search, except that which is shown above. Automated checking will be used during marking, and therefore the output must be precise.
- **Should** call the function `test()` to perform any assertion testing etc.

Extension

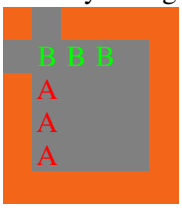
Basic assignment = 90%. Extension = 10%.
If you'd like to try an extension, make sure to submit *extension.c* and a brief description in a

extension.txt file. This could involve a faster search technique, better graphical display, user input or something else of your choosing.

■

8.5 Car Park

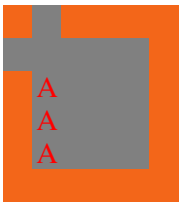
In a 2-dimesional car park as seen from above, vehicles are allowed to park either vertically or horizontally on a grid :



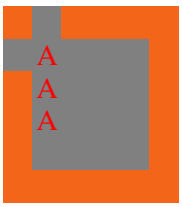
Here there are two vehicles, both of length 3. One is vertical (aligned north/south) coloured red, and one is horizontal (aligned west/east) coloured green. Empty cells of the gridded car park are shown in gray, and immovable boundaries (bollards) are shown in orange. Vehicles can only move forwards or backwards one square at a time (vertical vehicles move up and down, horizontal vehicles move left and right). These moves cannot be into, or across, another vehicle, nor into a bollard.

Our aim is to get all the vehicles safely out of the car park. During a turn, one vehicle is allowed to move forwards or backwards one square. Once the front of a vehicle touches an empty square on the edge of the car park, it is deemed to have exited the car park, and removed.

To 'solve' the car park above, we would move vehicle B one move left (so that it exits) :



and then vehicle A up one :



and then up one more move so that it too exits :



All vehicles have now exited the car park taking a total of three turns.

Exercise 8.5.1 Write a program that reads in a car park file (specified on the command line), and shows the 'turns' to solve it. The file for the car park above looks like :

```
6x6
#.####
```

```
.BBB.#
#A...#
#A...#
#A...#
######
```

The first line has two numbers; the height of the car park (number of rows) and then the width (number of columns).
In the remainder of the file, vehicles are shown as a capital letter, gaps as a full-stop and bollards as a hash symbol. Each cars may only lie in the grid vertically or horizontally, and must be of at least length 2. Each vehicle must have a unique uppercase letter, the first of which must be an 'A', the next one be 'B' and so on.
We wil use a brute-force algorithm for searching over all moves for a solution :

1. You will use an array (list) of structures, each one containing the data for one car park. Note that you may choose to store the state of each car park, not as a 2D array, but as something that is easier to manipulate, e.g. an array of vehicles, including their position, orientation and whether they've exited or not. Each approach has pros and cons.
2. Put the initial car park into the front of this list, `f=0`.
3. Consider the car park at the **front** of the list (index `f`).
4. For this (parent) car park, find the resulting (child) car parks which can be created from all the possible vehicle moves. For each of these child car parks:
   - If this car park is unique (i.e. it has not been seen before in the list), add it to the end of the list.
   - If it has been seen before (a duplicate) ignore it.
   - If it is the 'final' car park, stop and print the solution.
5. Add one to f. If there are more car parks in the list, go to step 3.

To help with printing out the correct moves, when a solution has been found, each structure in the list will need to contain (amongst other things) a car park and a record of its parent car park, i.e. the car park that it was created from. Since you're using an array, this could simply be which index of the array was the parent.
The program reads the name of the car park definition file from the command line. If it finds a successful solution, it prints out the number of car parks that would be printed in the solution and **nothing else**, or else exactly the phrase 'No Solution?" if none can be found (as might be the case if it is impossible, or you simply run out of memory) :

```
$ ./carpark ../Git/Data/CarPark/6x6_2c_3t.prk
3 moves
$ .car/park ../Git/Data/CarPark/11x9_10c_26t.prk
26 moves
```

If the 'show' flag is used, your program will print out the solution in the correct order :

```
$ ./carpark -show ../Git/Data/CarPark/6x6_2c_3t.prk
#.####
.BBB.#
#A...#
#A...#
#A...#
######
```

```
#.####
.....#
#A...#
#A...#
#A...#
######

#.####
.A...#
#A...#
#A...#
#....#
######

#.####
.....#
#....#
#....#
#....#
######

3 moves
```

Your program :

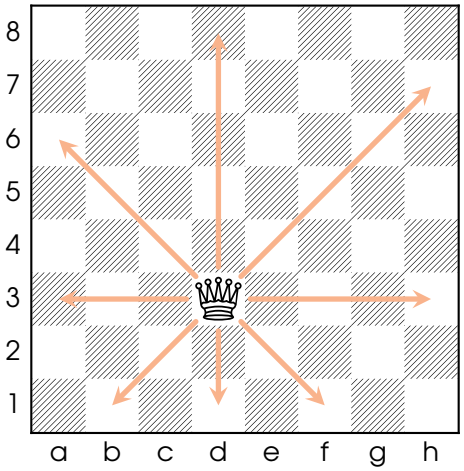
- **Must** use the algorithm detailed above (which is similar to a queue and therefore a breadth-first search). Do not use the other algorithms possible (e.g. best-first, guided, recursive etc.); the quality of your coding is being assessed against others taking the same approach.
- **Must not** use dynamic arrays or linked lists. Since car parks cannot be any larger than 20×20, you can create car parks of this size, and only use a sub-part of them if the car park required is smaller. The list of car parks can be a fixed (large) size.
- **Should** check for invalid car park definition files, and report in a graceful way if there is a problem, aborting with `exit(EXIT_FAILURE)` if so.
- **Should not** print anything else out to screen after successfully completing the search, except that which is shown above. Automated checking will be used during marking, and therefore the output must be precise.
- **Should** call the function `test()` to perform any assertion testing etc.

Extension

Basic assignment = 90%. Extension = 10%.
If you'd like to try an extension, make sure to submit *extension.c* and a brief description in a *extension.txt* file, and an *extension.mak* Makefile, allowing me to build your code using `make extension`. The extension could involve a faster search technique, better graphical display, user input or something else of your choosing. ■

8.6 The N-Queens Puzzle

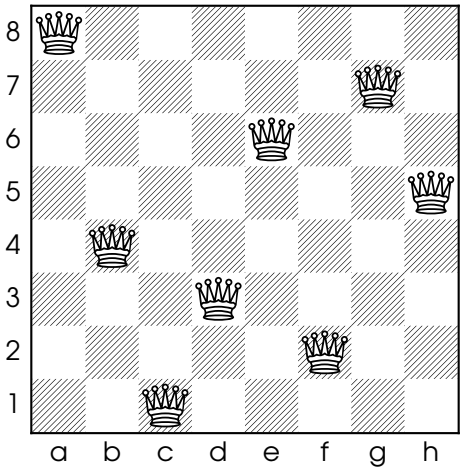
In the game chess, the piece known as the Queen can take other pieces horizontally, vertically or diagonally :



The eight queens puzzle is the problem of placing eight chess queens on an 8×8 chessboard so that no two queens threaten each other; thus, a solution requires that no two queens share the same row, column, or diagonal.

www `https://en.wikipedia.org/wiki/Eight_queens_puzzle`

One possible solution for an 8×8 board is :



Since there must be one, and one only, queen in each file (column), the above board can be summarized using the queen's position on each rank (height) - the board above would be numbered 84136275.
Since finding all solutions may take a long time, it is normal to allow for board sizes other than 8×8 (but always square) by specifying the size of the board, n.

Exercise 8.6.1 Write a program to find solutions to the n queens problem, using a brute-force algorithm to find all possible solutions. You will use an array (list) of structures, each one containing the data for one board :

1. Put the initial (empty) board into the front of this list, `f=0`.
2. Consider the board at the **front** of the list (index `f`).

3. For this (parent) board, find the resulting (child) boards which can be created by placing one queen into the board into a position that doesn't check any other queens. For each of these child boards :
   - If a child is unique (i.e. it has not been seen before in the list), add it to the end of the list.
   - If it has been seen before (a duplicate) ignore it.
   - If it is a solution board, keep a record of the number of them (and possibly) print out a summary of this board.
4. Add one to f. If there are more boards in the list, go to step 2.

We will allow for board sizes other than 8×8, but **never** greater than 10×10, as specified on the command line :

```
$ ./8q 6
4 solutions
```

If the 'verbose' flag is used, your program will also print out summaries of each solution :

```
$ ./8q -verbose 6
362514
246135
531642
415263
4 solutions
```

(Since 10×10 boards may be printed, we'll use the numbers $1 \ldots 9$ and also A).
Your program :

- **Must** use the algorithm detailed above (which is similar to a queue and therefore a breadth-first search). Do not use the other algorithms possible (e.g. best-first, permutations, etc.); the quality of your coding is being assessed against others taking the same approach.
- **Must not** use dynamic arrays or linked lists. Since boards cannot be any larger than 10×10, you can create boards of this size, and only use a sub-part of them if the board required is smaller. The list of boards can be a fixed (large) size.
- **Should** check for invalid board sizes specified, and report in a graceful way if there is a problem, aborting with `exit(EXIT_FAILURE)` if so.
- **Should not** print anything else out to screen after successfully completing the search, except that which is shown above. Automated checking will be used during marking, and therefore the output must be precise.
- **Should** call the function `test()` to perform any assertion testing etc.
- **Should** be submitted by uploading a single `8q.zip` file to Blackboard, which will include (at least) `8q.c` and `Makefile` which allows me to compile your code by typing `make 8q` which creates `8q`, the executable file and can be run using `make run`.

Extension

Basic assignment = 90%. Extension = 10%.
If you'd like to try an extension, make sure to put *extension.c* and an explanation of it in *extension.txt* as part of your submission. Update your `Makefile` so that we can build your code using `make extension` and run it with `make extrun`. The extension could involve a faster search technique, better graphical display, user input or something else of your choosing.

■

8.7 Match Drop

The puzzle *Match Drop* consists of a 2*D* rectangular grid of tiles, all of which are labelled with an upper-case letter e.g. '*A*', '*B*... '*Z*'. Outside of this grid is another tile, known as the 'hawk' tile:

A
A B C
A B C
C B A

The 'hawk' tile can be used to push down one column. The hawk becomes the top tile in this column, and the bottom tile of this column becomes the new hawk tile. The task is to roll one column at a time, so that every column contains the same letters.
In the above example, if the hawk tile is played to push down the first column, then the new board now looks like:

C
A B C
A B C
A B A

Both the first and seond columns are now completed and are never altered again. Using the hawk on column three produces:

A
A B C
A B C
A B C

and our search for a finshed (completed) board is over.

Exercise 8.7.1 Write the functions specified in `md.h` that allows a board file to be read in, and computes the number of moves required to solve it.
You may assume that the maximum height and width of a board is 6.
The brute-force algorithm for searching over all moves for a solution goes like this :

1. You will use an `alloc()`'d array (list) of boards.
2. Put the initial board into the front of this list, `f=0`.
3. Consider the board at the **front** of the list (index `f`).
4. For this (parent) board, find the resulting (child) boards which can be created from all the possible column pushes (already completed columns are not altered). For each of these child boards:
   - If this board is unique (i.e. it has not been seen before in the list), add it to the end of the list.
   - If it has been seen before (it's a duplicate) ignore it.
   - If it is the 'final' board, stop and (possibly, print the solution).
5. Add one to f. If there are more boards in the list, go to step 3.

To help with printing out the correct moves, when a solution has been found, each board in the list will need to contain (amongst other things) a 2D grid of tiles, the hawk, and a record of its parent board, i.e. the board that it was created from. Since you're using an array, this could simply be the index of the array that was the parent.
Your program :

- **Must** use the algorithm detailed above (which is similar to a queue and therefore a breadth-first search). Do not use the other algorithms possible (e.g. best-first, guided, recursive etc.); the quality of your coding is being assessed against others taking the same approach, and if you do something different it won't get any marks.

- **Must not** use dynamic arrays or linked lists. Since boards cannot be any larger than 6×6, you can create boards of this size, and only use a sub-part of them if the board required is smaller. The list of boards can be a fixed (large) size (maybe $200,000$?)
- **Should** be able to cope with invalid board definition files with a graceful exit.
- **Should not** print anything out to screen after successfully completing the search, except when in verbose mode. Automated checking will be used during marking, and therefore the output must be very precise. For the `driver.c` file given, the verbose output is required for `2moves.brd`, for which the verbose flag has been set in the solve function. In this case, the output will look like:

```
% ./md
ABC
ABC
ABC
CBA

ABC
ABC
ABC
ABA

ABC
ABC
ABC
ABC
```

- **Should** call the function `test()` to perform any assertion testing etc.

■

8.8 NumMatch

The game *Number Match* consists of a $2D$ rectangular grid of numbers. The aim is to remove all of the numbers from the grid by finding 'matches' for pairs of numbers, which are either the same (e.g. a 4 with a 4), or which total to ten (e.g. a 3 with a 7). Pairs also need to be connected by being adjacent to each other (8-connected), or else have no other numbers between them, so in the following grid:

2	2	8	1	2
6	8	7	2	8
5	4	7	5	9
9	4	6	9	2

we could match a 7 with a 7:

2	2	8	1	2
6	8	·	2	8
5	4	·	5	9
9	4	6	9	2

opening up the possibility of matching the 8 with the 2:

2	2	8	1	2
6	·	·	·	8
5	4	·	5	9
9	4	6	9	2

If we now match the 4 with the 4:

2	2	8	1	2
6	·	·	·	8
5	·	·	5	9
9	·	6	9	2

then this frees up the 'long' diagonal match of the 1 with the 9:

2	2	8	·	2
6	·	·	·	8
5	·	·	5	9
·	·	6	9	2

If you are careful with the order in which you do the matching, it's possible to match all of the numbers.

Exercise 8.8.1 Write the functions specified in `nm.h` that allow a board to be created by filling the board with random numbers from $1 \ldots 9$ (`randfill()`), `take()` to remove valid pairs, and `solve()` which decides whether the board is solvable or not.
You may assume that the height of the board is 4 cells, and the width is 5.
The brute-force algorithm for searching over all moves for a solution goes like this :

1. You will use a large array (list) of boards.
2. Put the initial board into the front of this list, `f=0`.
3. Consider the board at the **front** of the list (index `f`).
4. For this (parent) board, find the resulting (child) boards which can be created from all the possible valid matches of number pairs. For each of these child boards:
   - If this board is unique (i.e. it has not been seen before in the list), add it to the end of the list.
   - If it has been seen before (it's a duplicate) ignore it.
   - If it is the 'final' board, stop.
5. Add one to f. If there are more boards in the list, go to step 3.

Your program :

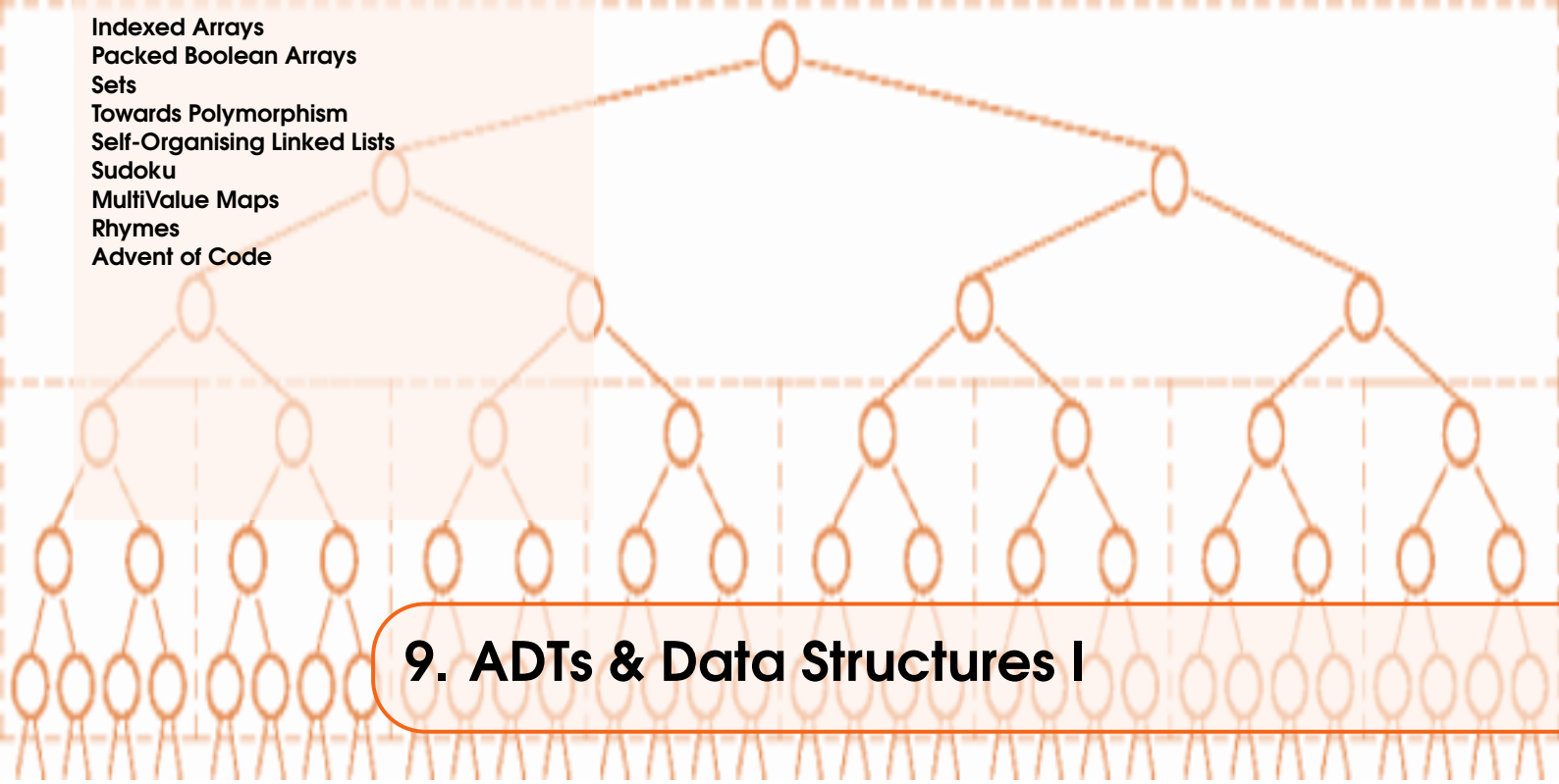
- **Must** use the algorithm detailed above (which is similar to a queue and therefore a breadth-first search). Do not use the other algorithms possible (e.g. best-first, guided, recursive etc.); the quality of your coding is being assessed against others taking the same approach, and if you do something different it won't get any marks.
- **Must not** use dynamic arrays or linked lists. The list of boards can be a fixed (large) size ($100,000$?)
- **Should not** print anything out to screen after successfully completing the search.

- **Should** call implement the function `test()` to perform any assertion testing, (including tests of any of your helper functions).
- Should run silently (except in the case of an `EXIT_FAILURE`).

Extension

Basic assignment = 90%. Extension = 10%.
If you get your program working, there's a small extension available. Recode your program so that it executes as quickly as possible in files called `ext.c` and `ext_mydefs.h`. For this I will compute the time taken for your code to solve a series of complex boards using `/usr/bin/time`. ■

Indexed Arrays
Packed Boolean Arrays
Sets
Towards Polymorphism
Self-Organising Linked Lists
Sudoku
MultiValue Maps
Rhymes
Advent of Code



9. ADTs & Data Structures I

9.1 Indexed Arrays

In the usual places are the files `arr.h`, `testarr.c` and a Makefile. My files `general.c` and `general.h` (if you want to use them) are in the ADT section of the github site. You will write `Realloc/realloc.c` and `Realloc/specific.h`. These files will allow you to build, and test, the ADT for a 1D Indexed Array. This simple replacement for C arrays is 'safe' in the sense that if you write the array out-of-bounds, it will be automatically resized correctly (using `realloc()`). The interface to this ADT is in `arr.h` and its implementation will be in `Realloc/realloc.c` Typing `make`, compiles these files together with the test file `testarr.c`. Executing `./testarr` should result in all tests passing correctly.

```
% make run
Basic Array Tests ... Start
Basic Array Tests ... Stop
```

Exercise 9.1.1 Write `Realloc/realloc.c` nd `specific.h` so that the functionality in the driver code `testarr.c` is implemented. ■

9.2 Packed Boolean Arrays

In C99, every individual variable must have a different address. For this reason, a `bool` must take up a least one byte since this is the smallest individual element uniquely addressable by most hardware.

In some cases it would be good to have a datatype that truly uses one bit of storage for each Boolean. One such example is known as the *packed* Boolean array, where an array of 64 elements takes 8 bytes.

Here we create an ADT for packed Boolean arrays, allowing logical operations to be performed on an entire array, and individual bits to be set and unset.
This will involve manipulation of the bits of the data using the C bitwise logical operators 'xor' (^), 'or' (|) and 'and' (&).

www `https://en.wikipedia.org/wiki/Bitwise_operation`

Exercise 9.2.1 Given the files `testboolarr.c` and `boolarr.h`, complete the Boolean Array ADT using a reallocating array of unsigned chars. You will need to write both `realloc.c`, which will define the functions in boolarr.h, and `specific.h`, which will contain your header information. Ignoring the overhead of the main structure, which might store the capacity of the underlying array, and the number of valid bits in use, the array should be around `nbits`/8 bytes in length. ■

9.3 Sets

Sets are an important concept in Computer Science. They enable the storage of elements (members), guaranteeing that no element appears more than once. Operations on sets include initializing them, copying them, inserting an element, returning their size (cardinality), finding if they contain a particular element, removing an element if it exists, and removing one element from a random position (since sets have no particular ordering, this could be the first element). Other set operations include union (combining two sets to include all elements), and intersection (the set containing elements common to both sets).

www `https://www.mathsisfun.com/sets/sets-introduction.html`

www `https://en.wikipedia.org/wiki/Set_(mathematics)`

The definition of a Set ADT is given in `set.h`, and a file to test it is given in `testset.c`.

Exercise 9.3.1 Write an implementation of the Set ADT which builds on top of the Indexed Array ADT introduced in Exercise 9.1.1 and which compiles againset `testset.c`. ■

9.4 Towards Polymorphism

Polymorphism is the concept of writing functions (or ADTs), without needing to specify which particular type is being used/stored. To understand the quicksort algorithm, for instance, doesn't really require you to know whether you're using integers, doubles or some other type. C is not very good at dealing with polymorphism - you'd need something like Python, Java or C++ for that. However, it does allow the use of `void*` pointers for us to approximate it.

Exercise 9.4.1 Extend the array ADT discussed in Exercise 9.1.1, so that any type can be used - files `varr.h` and `testvarr.c` are available in the usual place - use the Makefile used there, simply swapping `arr` for `varr` at the top. ■

9.5 Self-Organising Linked Lists

A self-organising linked list (SOLL) improves search efficiency (over an unsorted list) by rearranging the elements in the list each time they are accessed:

www `https://en.wikipedia.org/wiki/Self-organizing_list`

If no self-organisation is done, a SOLL behaves similarly to a collection. New elements are inserted at the end of the list, and searching is done from the start to the end using pointer-chasing. For efficiencies sake, we keep pointers to both the start and the end of the list. Insertion, therefore, has a $O(1)$ cost.

However, if self-organisation is done via the *move-to-front* (MFT) policy, each time an element is accessed, it is moved to the start of the list. This means that the next time we search for the same element, it will be found more quickly, since it is near the start of the list. Once the element is found, this re-ordering also has a complexity of $O(1)$.

The MFT policy can sometimes be too aggressive; uncommon elements that are searched for will move to the front, potentially displacing elements that are being searched for more frequently. An alternative is the *transpose* policy. When an element is accessed, it is moved one place in the list closer to the start. This once again can be done in constant time, providing we keep a copy of a pointer to the previous element during the search, or alternatively, use a doubly-linked list e.g. each element has both a next and a previous pointer.

Exercise 9.5.1 Implement a SOLL using linked lists to implement these different policies and store strings.

- 55% Write the files `Linked/specific.h` and `Linked/linked.c`. Using the `soll.mak` makefile, you can compile this against the `soll.h` file and one of the test/driver files `testsoll.c`, `build.c` or `uniq.c`.
  The ADT should implement the standard functions: `soll_init()`, `soll_add()`, `soll_remove()`, `soll_isin()`, `soll_tostring()`, `soll_size()` and `soll_free()`. In addition, write the function `soll_freq()` which reports the frequency of access for a particular element. Therefore, each element (structure) of the list has the overhead of keeping track of the number of times it has been accessed.
- 30% Show a testing strategy on the above by submitting `testing.txt` where you give details of unit testing, white/black-box testing done on your code. Describe any test-harnesses used. Convince me that every line of your C code has been tested.
- 15% Show an extension to the project in a direction of your choice via `extension.txt`. It should demonstrate your **understanding** of some aspect of programming or S/W engineering. Make sure it's clear what has been done, why, and how to compile it.

■

9.6 Sudoku

This concerns the automated solving of one of the world's most famous puzzles, Sudoku:



`https://en.wikipedia.org/wiki/Sudoku`

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Exercise 9.6.1 Using the simple ADT framework provided and the constraint propagation approach described in:

www `https://norvig.com/sudoku.html`

write a program that can solve "easy" puzzles. The constraint propagation approach cannot completely solve "hard" puzzles. A basic ADT is provided for you in `sudoku.h`, along with a simple test file `testsudoku.c`. Write `Fixed/fixed.c` and `Fixed/specific.h`, so that:

```
% make run
Basic Sudoku Tests ... Start
Basic Sudoku Tests ... Stop
Basic Sudoku Tests ... Start
Basic Sudoku Tests ... Stop
```

works correctly.
The code will hard-wire the grid to be a fixed size $2D$ array, and be capable of reading in the different files present in `Data/Sudoku`. Submit a single `.zip` file which contains your entire submission, including `Fixed/fixed.c`, `Fixed/specific.h` and a `Makefile` (if different to the one provided). In addition, if you wish, you can provide extra material (extra testing etc.). If so provide a `basic.txt` file explaining what you've done. ■

Exercise 9.6.2 Extend your program so that all grids are solvable, even the "hard" examples given. One typical way to do this is to make a guess for a square, and then backtrack to undo this if it leads to an invalid solution when using constraint propagation. Efficiency of your solution is not a concern here - focus on creating beautifully crafted (and tested) code.
Submit a single `.zip` file which contains your entire submission, including the updated versions of `Fixed/fixed.c` and `Fixed/specific.h` in addition to `solution.txt` which briefly describes the algorithm used. Include a `Makefile` so that `make hard` builds and runs the code.

■

Exercise 9.6.3 Extend your program in a manner of your own choosing - this could be a faster approach to solving all grids (but maybe at the expense of code readability), exploring Sudokus larger than $9x9$, or allowing better I/O or user-interaction.
Submit a single `.zip` file which contains your entire submission, including the updated versions of `Fixed/fixed.c` and `Fixed/specific.h` in addition to `extend.txt` which briefly describes what you have done. Include a `Makefile` so that `make extension` builds and runs the code.

■

9.7 MultiValue Maps

Many data types concern a single value (e.g. a hash table), so that a string (say) acts as both the key (by which we search for the data) and also as the object we need to store (the value). An example of this a spelling checker, where one word is stored (and searched for) at a time. However, sometimes there is a need to store a value based on a particular key - for instance an associative array in Python allows you to perform operations such as :

```
population["Bristol"] = 536000
```

where a value (the number 536000) is stored using the key (the string "Bristol"). One decision you need to make when designing such a data type is whether multiple values are allowed for the same key; in the above example this would make no sense - Bristol can only have one population size. But if you wanted to store people as the key, with their salary as the value, you might need to use a MultiValue Map (MVM) since people can have more than one job.

Here we write the abstract type for a MultiValueMap that stores key-value pairs, where both the key and the value are strings.

Exercise 9.7.1 The definition of an MVM ADT is given in `mvm.h`, and a file to test it is given in `testmvm.c`. Write `mvm.c`, so that:

```
% make -f mvm_adt.mk
./testmvm
Basic MVM Tests ... Start
Basic MVM Tests ... Stop
```

works correctly. Use a simple linked list for this, inserting new items at the head of the list. Make no changes to any of my files. ■

9.8 Rhymes

In the usual place is a dictionary which, for every word, lists the phonemes (a code for a distinct unit of sound) used to pronounce that word in American English. In this file the word itself, and its phonemes, are separated by a '#'). For instance:

```
BOY#B OY1
```

shows that the word `BOY` has two phonemes : `B` and `OY1`.

A simple attempt at finding rhymes for boy would match every word that has `OY1` as its final phoneme. This gives you:

```
POLLOI MCVOY LAFOY ALROY ILLINOIS CROIX DECOY REDEPLOY CLOY
LAVOY MOYE LOYE STOY PLOY KNOY EMPLOY ELROY JOY COY LACROIX
DEVROY ENJOY LOY COYE FOYE MOY DOI BROY TOY LABOY ROI HOY
ROYE NEU CROY SOY YOY MCCOY CHOY GOY ROY BOLSHOI MALLOY JOYE
DESTROY DELACROIX(1) DEBOY MCROY CHOI UNDEREMPLOY FLOY MCKOY
TOYE AHOY BOY OYE SGROI FOIE(1) TROY DEPLOY SAVOY UNEMPLOY
SCHEU WOY BOYE HOYE FOY OI HOI KROY EMPLOY(1) FLOURNOY OIE
MCCLOY ANNOY OY DEJOY
```

Using two phonemes to do the matching is too many, since the only matches are for words that have exactly the same pronunciation:

```
LABOY DEBOY BOY BOYE
```

which are not really rhymes, but homophones. Therefore, using the **correct** number of phonemes will be key to finding 'good' rhyming words.

Here we will use the MutliValue Map written in Exercise 9.7.1 to create two maps. An MVM `map1` stores the word (as the key) and its final *n* phonemes as a single string (the value). Now `map2` stores the word (value), keyed by its final *n* phonemes as a single string. Looking up the phonemes associated with a word can be done using the word as a key [1] (via `map1`), and looking up a word given its phonemes can be achieved using `map2`.

[1] Strictly speaking, we don't need the *map*1 to be capable of storing multiple values, since every word in the dictionary is unique. We'll use it here though for simplicity.

Exercise 9.8.1 Read in the dictionary, and for each line in turn, store the data in the two maps. Now for a requested word to be 'rhymed', search for its phonemes using `map1` and then search for matching words using `map2`. The number of phonemes specified for this rhyming is given via the command line, as are the words to be rhymed:

```
$ ./homophones -n 3 RHYME
RHYME (R AY1 M): PRIME RHYME ANTICRIME(1) CRIME ANTICRIME
GRIME RIME
```

The `-n` flag specifies the number of phonemes to use (you may assume the number associated with it always is always separated from the flag by a space). If no `-n` flag is given then the value 3 is assumed. It only makes sense to use a value of $n \leq$ the number of phonemes in the two words being checked. If you use a value greater than this, the results are undefined.

The list of words to be matched may be found on the command line:

```
$ ./homophones -n 4 CHRISTMAS PROGRAM PASSING
CHRISTMAS (S M AH0 S): ISTHMUS CHRISTMAS CHRISTMAS
```

Use the Makefile supplied for this task. Make the output as similar to that shown above as possible. ■

9.9 Advent of Code

Have a look through the archives of the rather wonderful 'Advent of Code' website, which is a good place to begin your lifelong love of recreational programming.

www `https://adventofcode.com`

and find a simple puzzle to solve.

Exercise 9.9.1 Choose one of the simpler puzzles and solve it. ■



Depth
Two Trees
Binary Tree Visualisation
Lowest Common Ancestor
Huffman Encoding
Faster MVMs
Double Hashing
Separate Chaining

10. Trees & Hashing

10.1 Depth

The following program builds a binary tree at random:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <assert.h>
#include <time.h>

#define STRSIZE 5000

struct node{
        char c;
        struct node *left;
        struct node *right;
};
typedef struct node Node;

Node *MakeNode(char c);
void InsertRandom(Node *t, Node *n);
char *PrintTree(Node *t);

int main(void)
{

    char c;
    Node *head = MakeNode('A');
    Node *n;

    srand(time(NULL));
    for(c = 'B'; c < 'G'; c++){
        n = MakeNode(c);
        InsertRandom(head, n);
    }
    printf("%s\n", PrintTree(head));
    return 0;
```

```
}

Node *MakeNode(char c)
{

    Node *n = (Node *)calloc(1, sizeof(Node));
    assert(n != NULL);
    n->c = c;
    return n;

}

void InsertRandom(Node *t, Node *n)
{

    if((rand()%2) == 0){ /* Left */
        if(t->left == NULL){
            t->left = n;
        }
        else{
            InsertRandom(t->left, n);
        }
    }
    else{ /* Right */
        if(t->right == NULL){
            t->right = n;
        }
        else{
            InsertRandom(t->right, n);
        }
    }

}

char *PrintTree(Node *t)
{

    char *str;

    assert((str = calloc(STRSIZE, sizeof(char))) != NULL);
    if(t == NULL){
        strcpy(str, "*");
        return str;
    }
    sprintf(str, "%c (%s) (%s)", t->c, PrintTree(t->left), PrintTree(t->right));
    return str;

}
```

Each node of the tree contains one of the characters 'A' ... 'F'. At the end, the tree is printed out in the manner described in the course lectures.

Exercise 10.1.1 Adapt the code so that the maximum depth of the tree is computed using a recursive function. The maximum depth of the tree is the longest path from the root to a leaf. The depth of a tree containing one node is 1.

■

10.2 Two Trees

Adapt the code shown in Exercise 10.1, so that two random trees are generated.

Exercise 10.2.1 Write a Boolean function that checks whether the two trees are identical or not. ■

10.3 Binary Tree Visualisation

The course notes showed a simple way to print out integer binary trees in this form :

```
20(10(5(*)(*))(17(*)(*)))(30(21(*)(*))(*))
```

You could also imagine doing the reverse operation, that is reading in a tree in the form above and displaying it in a 'friendlier' style :

```
20----30
|     |
10-17 21
|
5
```

The tree has left branches vertically down the page and right branches horizontally right. Another example is :

```
17(2(*)(3(*)(4(*)(*))))(6(8(*)(*))(*))
```

which is displayed as:

```
17----6
|     |
2-3-4 8
```

The above examples show the most 'compact' form of displaying the trees, but you can use simplifying assumptions if you wish:

- The integers stored in the tree are always ≥ 0.
- The integers stored in the tree are 5 characters (or less) in length.
- It is just as valid to print the tree in either of these ways :

```
1-6        00001-00006          00001--------------00006
| |          |     |              |                    |
2 7        00002 00008          00002                00007
|            |                    |
3-4-5      00003-00004-00005    00003-00004-00005
```

Exercise 10.3.1 Write a program that reads in a tree using `argv[1]` and the tree displayed to `stdout` with no other printing if no error has occurred. ■

Exercise 10.3.2 Write a program that reads in a tree using `argv[1]` and displays the tree using SDL. ■

10.4 Lowest Common Ancestor



UNDER CONSTRUCTION - Please don't attempt this yet.
We'll have this new exciting exercise ready for you soon!

Which node in a tree is the lowest common ancestor of two others ?

10.5 Huffman Encoding

Huffman encoding is commonly used for data compression. Based on the frequency of occurrence of characters, you build a tree where rare characters appear at the bottom of the tree, and commonly occurring characters are near the top of the tree.

For an example input text file, a Huffman tree might look something like:

```
010 :        00101 (   5 *  125)
' ' :          110 (   3 *  792)
'"' :    111001010 (   9 *   12)
''' :     00100000 (   8 *   15)
'(' :  01100000100 (  11 *    2)
')' :  01100001101 (  11 *    2)
',' :      1001001 (   7 *   39)
'-' :      0010010 (   7 *   31)
'.' :      1001100 (   7 *   40)
'/' :  00100110000 (  11 *    1)
'0' :  11100110010 (  11 *    3)
'1' :     00100010 (   8 *   15)
'3' :  01100000101 (  11 *    2)
'4' :  01100001001 (  11 *    2)
'5' :  11100110011 (  11 *    3)
'6' :  01100001000 (  11 *    2)
'7' :  01100001100 (  11 *    2)
'8' :    001001101 (   9 *    8)
'9' :     10010000 (   8 *   18)
':' :  01100001011 (  11 *    2)
'A' :     00100111 (   8 *   16)
'B' :    111001101 (   9 *   13)
'C' :     10011011 (   8 *   22)
'D' :    111001110 (   9 *   13)
'E' :     10011010 (   8 *   19)
'F' :    111001000 (   9 *   11)
'G' :   0110000000 (  10 *    4)
'H' :   1110011111 (  10 *    7)
'I' :   1110010011 (  10 *    6)
'J' :  11100111101 (  11 *    3)
'K' :   1110010111 (  10 *    6)
'L' :     00100011 (   8 *   15)
'M' :  11100111100 (  11 *    3)
'N' :  01100001010 (  11 *    2)
'O' :  01100000111 (  11 *    2)
'P' :   1110011000 (  10 *    6)
'R' :   0110000111 (  10 *    5)
'S' :     10010001 (   8 *   19)
'T' :   0010011001 (  10 *    4)
```

```
'U' :   1110010010 (  10 *    5)
'W' :   0110000001 (  10 *    4)
'a' :         1010 (   4 *  339)
'b' :      1111110 (   7 *   60)
'c' :       100101 (   6 *   77)
'd' :        01101 (   5 *  143)
'e' :          000 (   3 *  473)
'f' :       100111 (   6 *   84)
'g' :       111000 (   6 *   94)
'h' :        11110 (   5 *  223)
'i' :         0100 (   4 *  266)
'j' :  01100000110 (  11 *    2)
'k' :     00100001 (   8 *   15)
'l' :        10110 (   5 *  176)
'm' :       101111 (   6 *   92)
'n' :         0111 (   4 *  288)
'o' :         0101 (   4 *  269)
'p' :       101110 (   6 *   89)
'q' :  00100110001 (  11 *    2)
'r' :        11101 (   5 *  214)
's' :         0011 (   4 *  260)
't' :         1000 (   4 *  305)
'u' :       111110 (   6 *  108)
'v' :      0110001 (   7 *   37)
'w' :      1111111 (   7 *   60)
'x' :   1110010110 (  10 *    6)
'y' :       011001 (   6 *   72)
2916 bytes
```

Each character is shown, along with its Huffman bit-pattern, the length of the bit-pattern and the frequency of occurrence. At the bottom, the total number of bytes required to compress the file is displayed.

Exercise 10.5.1 Write a program that reads in a file (argv[1]) and, based on the characters it contains, computes the Huffman tree, displaying it as above. ■

10.6 Faster MVMs

The ADT for MVMs used in Exercise 9.7.1 is a simple linked list - insertion is fast, but searching is slow.

Exercise 10.6.1 Write a new version of this MVM ADT called `fmvm.c` that implements exactly the same functionality but has a faster search time. The file `fmvm.h` will change very little from `mvm.h`, with maybe only the structures changing, but not the function prototypes. A similar testing file to that used previously, now called `testfmvm.c` should also be written. Note that any ordering of data when using the `mvm_print` and `mvm_multisearch` functions is acceptable, so these can't be tested in exactly the same manner.

By simply changing the `#include` from `<mvm.h>` to `<fmvm.h>` in your `homephones.c` file from Exercise 9.8.1, and compiling it against `fmvm.c`, I can test that program works identically.

Make it clear what you have done to speed up your searching (and how) using comments at the top of `fmvm.h`

Submit : `fmvm.c`, `fmvm.h` and `testfmvm.c` ■

10.7 Double Hashing

Here we use double hashing, a technique for resolving collisions in a hash table.

Exercise 10.7.1 Use double hashing to create a spelling checker, which reads in a dictionary file from `argv[1]`, and stores the words.

Make sure the program:

- Use double hashing to achieve this.
- Makes no assumptions about the maximum size of the dictionary files. Choose an initial (prime) array size, created via malloc(). If this gets more than 60% full, creates a new array, roughly twice the size (but still prime). Rehash all the words into this new array from the old one. This may need to be done many times as more and more words are added.
- Uses a hash, and double hash, function of your choosing.
- Once the hash table is built, reads another list of words from `argv[2]` and reports on the *average* number of look-ups required. A *perfect* hash will require exactly 1.0 look-up. Assuming the program works correctly, this number is the only output required from the program.

■

10.8 Separate Chaining

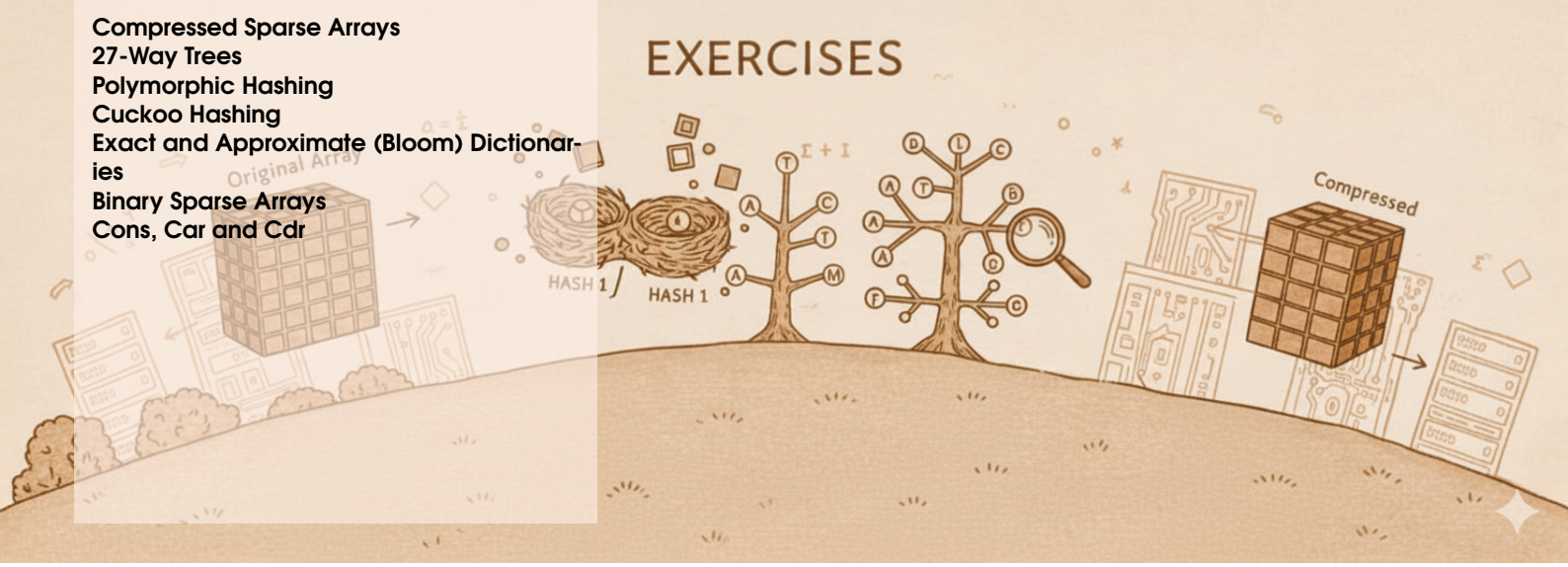
Separate chaining deals with collisions by forming (hopefully small) linked lists out from a base array.

Exercise 10.8.1 Adapt Exercise 10.7.1 so that:

- A linked-list style approach is used.
- No assumptions about the maximum size of the dictionary file is made.
- The same hash function as before is used.
- Once the hash table is built, reads another list of words from `argv[2]` and reports on the *average* number of look-ups required. A *perfect* hash will require exactly 1.0 look-up, on average. Assuming the program works correctly, this number is the only output required from the program.

■

Compressed Sparse Arrays
27-Way Trees
Polymorphic Hashing
Cuckoo Hashing
Exact and Approximate (Bloom) Dictionaries
Binary Sparse Arrays
Cons, Car and Cdr



11. ADTs II & Data Structures

11.1 Compressed Sparse Arrays

Sometimes it is known in advance that you need an array-type data structure, but that it will be populated very sparsely i.e. very few of the locations will be used. This is extrememly memory inefficient; using one large array for this would be wasteful, since most of the memory would be redundant. Here we investigate one of the many ways of creating an alternative, where we trade-off speed for memory: Compact Sparse Arrays (CSA) which aim to be memory efficient for sparsely occupied arrays.

The main part of the CSA is a 'block' - a way of storing the values (and corresponding indices) of a section of the array compactly. A block contains:

- A 64-bit mask (represented using the type `uint64_t`) showing which of the cells of the arrays are in use.
- a (`realloc()`'d) integer array storing the values, ordered by their index.
- an index offset (a multiple of 64) showing what part of the overall array that block represents e.g. the block that contains indices 0...63 would have an offset of 0, block that contains indices 64...127 would have an offset of 64, and so on.

The CSA structure consists of a `realloc()`'d array of blocks (ordered by their `offset`), and a counter, n, storing the total number of blocks in use.

Exercise 11.1.1 Write an ADT to implement the `csa.h` given. Write the source files `mydefs.h` and `csa.c` such that the driver files `driver.c` and `fibmemo.c` work correctly. Ensure that this all works with my `Makefile` which **must be** used **unaltered**. The basic operations are :

```
csa* csa_init(void);
bool csa_set(csa* c, int idx, int val);
bool csa_get(csa* c, int idx, int* n);
void csa_tostring(csa* c, char* s);
void csa_free(csa** l);
```

The function `csa_set()` allows an integer to be written to a particular index. If it's the first index for a particular block, this will trigger the creation of the memory for this block.

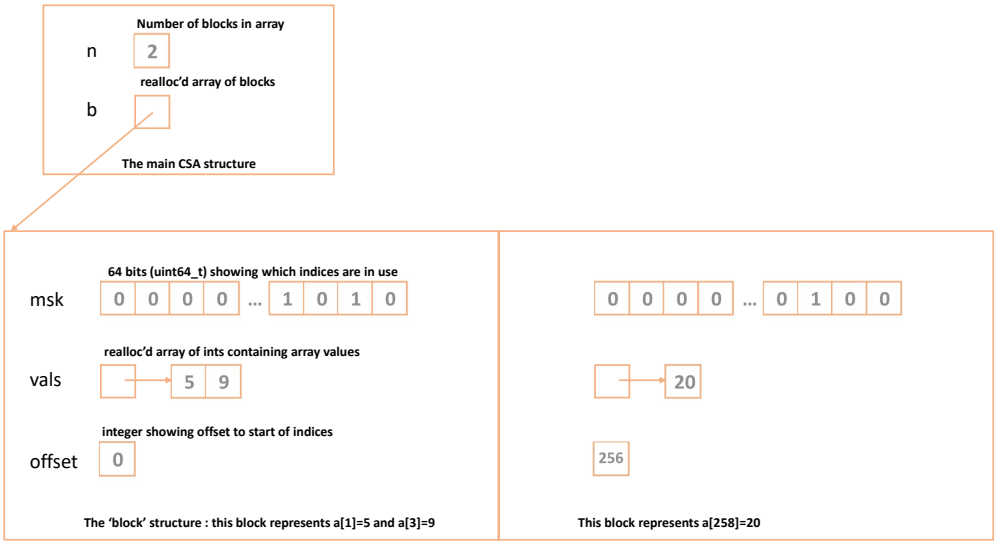


Figure 11.1: The Compressed Sparse Array. A dynamically allocated array of 'blocks' stores indices/values for the array, 64 values at a time.

`csa_get()` returns a Boolean showing whether `idx` is in use or not. If it is, the integer value is copied into n.
`csa_tostring()` allows a character based version of the CSA to be produced, showing each block in turn.

- You will use `realloc()` for the CSA - blocks are created only as and when required. No linked lists are used for this assignment.
- Even if you don't get all the functions to work correctly, make sure the code still compiles by writing 'dummy' functions as placeholders, even if some of the assertions fail.
- This assignment is brand new. If minor typos etc. are uncovered as you complete the assignment we may update this documentation and/or the online files provided. Please check regularly to make sure you are using the most recent version.

This is worth 80% **of the marks.**

Two extensions are available, each worth 10%.

```
void csa_foreach(void (*func)(int* p, int* acc), csa* c, int* ac);
```

Allows the user to pass a function, which is in turn passed a pointer to all the values in the array. This would allow the user to, for instance, sum up the values, double them all etc. The `driver.c` file shows some examples of this, and `isfactorial.c` uses this to find a range of factorials.

```
bool csa_delete(csa* c, int idx);
```

Array elements can be deleted, and memory freed up if blocks are no longer used. If all the elements are deleted, you don't really need `csa_free()` since all the blocks have been freed, and simply calling `free()` on the csa structure should be enough. This allows the file `sieve.c` to be used to compute a range of prime numbers.

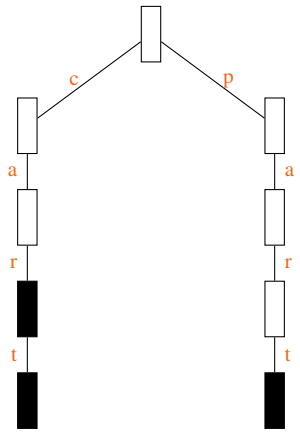
■

11.2 27-Way Trees

The Dictionary Abstract Data Type (ADT) allows words to be stored in such that they may be efficiently checked later, to ensure that words have been spelled correctly etc. There are many ways of implementing the Dictionary ADT (binary trees, hash tables and so on), but here we look at a tree structure which has a dynamic collection of nodes.
The 27-Way Tree has 27 downward links, each corresponding to one of the letters $a \ldots z$ and one to the apostrophe character '. This has many similarities with *tries*:

www `https://en.wikipedia.org/wiki/Trie`

Any word can be stored in the tree. You start at the top node, and, given the first character follow the relevant downward pointer. So, if the first character is 'a' you follow the first pointer down from the first node to another. If the next letter is 's' then you follow the 19th downwards pointer from that node to the following one. Each pointer corresponds to a letter (or the apostrophe), and each level of the tree corresponds to the length of a word.



In the above diagram, the words 'car', 'cart' and 'part' have been stored in such a 27-Way Tree. Since 'car' is a substring of 'cart' we have to find a way of making it clear where valid word ending are. In the diagram, black-filled nodes show such *terminal* nodes. The string 'ca' is not a valid word since the block immediately afterwards has not been marked as such. The word 'par' is not valid either, since it has never been added to the dictionary (but could, in principle, be added later).

Exercise 11.2.1 Write an ADT to implement the `t27.h` interface given. Write the source file `t27.c` such that the driver file `driver.c` works correctly. Ensure that this all works with the `Makefile` which, as with `t27.h` **must be** used **unaltered**. The standard operations are :

```
dict* dict_init(void);
bool dict_addword(dict* p, const char* str);
```

```
dict* dict_spell(const dict* p, const char* str);
int dict_nodecount(const dict* p);
int dict_wordcount(const dict* p);
int dict_mostcommon(const dict* p);
void dict_free(dict** p);
```

The function `dict_addword()` allows a string to be inserted into the tree, and the node after marked as a terminal node.
`dict_spell()` returns a pointer to the terminal node corresponding to the string (if it has been inserted) and `NULL` otherwise.
`dict_nodecount()` returns the total number of nodes which are part of the tree, and `dict_wordcount()` returns the total number of words which have been inserted into the tree **including** duplicates. A frequency counter is used in each node to keep a track of words which have been added multiple times. In this case this counter is incremented, but no new nodes are created.
`dict_mostcommon()` returns the number of times the most common word has been inserted.
The above functions are the standard assignment and are worth 60%. There are two additional advanced functions for you to implement, each worth up to 10% of the assignemnt:

```
unsigned dict_cmp(dict* p1, dict* p2);
```

which counts the least number of nodes you must traverse to move from one node to another in the tree. In the above figure, there are 7 steps from 'car' to 'part'. The other is:

```
void dict_autocomplete(const dict* p, const char* wd, char* ret);
```

which returns the substring corresponding to the additional letters required to get from the `wd` to the most frequent word below it. In the figure, to autocomplete 'car' would be the word 'cart', so the additional letters required are simply 't', which is stored in `ret`.

- We've provided the structure that **must** be used for this assignment. Do not attempt to change it. All functionality required is possible using this structure.
- For questions such as "but what about hyphenated words?" etc., look in the `driver.c` file. If we haven't tested for it in there, make a sensible decision as to what should happen. Provided that this file still operates correctly you can decide some of the edge-cases for yourself.
- Even if you don't get all the functions to work correctly, make sure the code still compiles by writing 'dummy' functions as placeholders, even if some of the assertions fail. I'll test the basic functionality separately from the two advanced ones.
- This assignment is brand new. If minor typos etc. are uncovered as you complete the assignment we may update this documentation and/or the online files provided. Please check regularly to make sure you are using the most recent version.

This is worth 80% **of the marks** (60% + 10% + 10%)

The manner in which you've been asked to implement the functionality above is **very** specific. However, exactly the same functionality (adding/spell-checking etc.) could be implemented in very different ways (linked lists, BSTs, hashing etc.). As an extension, write a 'rival' version to implement the standard functionality of these functions without using a 27-Way Tree. This should still ensure that some of the standard functionality in `driver.c` can be compiled against it. This means you can ignore `dict_autocomplete()`, `dict_cmp()` and

`dict_nodecount()` since these analyse something very specific about the 'shape' of the 27-Way Tree data structure. Use your own Makefile to achieve this, and put them in the sub-directory `Extension`.

If you do this, also submit `extension.txt` which details what you have done, the motivation for it, and how well it works in practice (or not), and comparing it with the original tree. This discussion (a few hundred words at most) is as (if not more) important than the code itself. You may wish to discuss which of the two methods is better, in terms of actual speed, complexity, memory usage, testability etc.
This worth 20% **of the marks.**

■

11.3 Polymorphic Hashing

Polymorphism is the concept of writing functions (or ADTs), without needing to specify which particular type is being used/stored. To understand the quicksort algorithm, for instance, doesn't really require you to know whether you're using integers, doubles or some other type. C is not very good at dealing with polymorphism - you'd need something like Java or C++ for that. However, it does allow the use of `void*` pointers for us to approximate it.

Exercise 11.3.1 Here we write an implementation of a polymorphic associative array using hashing, see `assoc.h`. Both the key & data to be used by the hash function are of unknown types, so we will simply store void pointers to both of these, and the user of the associative array (and **not** the ADT itself) will be responsible for creating and maintaining such memory, and also ensuring it doesn't change when in use by the associative array. In the `testassoc.c` file we show two uses for such a type : a simple string example (to find the longest word in English that is also a valid (but different) word when spelled backwards), and a simple integer example where we keep a record of how many unique random numbers in a range are chosen.

Since we do not know **what** the type of the key is, we need to be careful when comparing or copying keys. Therefore, in the `assoc_init(int keysize)` function, the user has to pass the size of the key used (e.g. `sizeof(int)`) or in the case or strings, the special value 0. Now we can use `memcmp()` and `memcpy()`, or in the case of strings, `strcmp()` and `strcpy()` for dealing with the keys. In the case of data, the ADT only ever needs to return a pointer to the data (not process it) via `assoc_lookup()`, so its size is not important.

Your hash table used to implement the ADT should be resizeable, and you may use open-addressing/double-hashing or separate chaining to deal with collisions. Make no assumptions about the maximum size of the array, and make the initial size of the array small e.g. 16 or 17 (a prime is useful if you're double hashing).. You can use any hash function you wish, but if it's off the internet (etc.) cite the source in a comment..

Submit a single `assoc.zip` file containing your code. Your standard submission will contain the directory structure, including the two files:
`Realloc/specific.h` and `Realloc/realloc.c`. I will use my `assoc.mk` Makefile to compile your code, so check that it works correctly. ■

Hint : when you do a resize you cannot simply copy the old table, but must rehash your data, one entry at a time, into the new table. This is because your hash function is based on the table size, and if the table size has changed you will be hashing keys into different locations.

11.4 Cuckoo Hashing

Exercise 11.4.1 Rewrite your associative array described in Exercise 11.3.1 using cuckoo hashing instead. This involves two tables, and a key is guaranteed to be found (if it has been inserted) in one of its two locations. Different hash functions must be used for each of the two tables. If you do the extension, add :
`Cuckoo/specific.h` and `Cuckoo/cuckoo.c` to your directory structure in `assoc.zip`. ■

11.5 Exact and Approximate (Bloom) Dictionaries

A dictionary ADT is given in `dict.h`. This allows words to be inserted, and also to check whether a word is already in the Dictionary or not. Each word in the Dictionary is unique (you don't store the same word multiple times).

Simple Spell Checkers can be made by feeding a 'white-list' of words to the Dictionary and then testing another list against these to find words that are misspelt.

Exercise 11.5.1 Implement the Dictionary using hashing and *Separate Chaining* for collision resolution. Write the files `Exact/specific.h` and `Exact/exact.c` to compile against my `dict.h` file and my two test/driver files `testdict.c` and `spelling.c`. There is no need to ever resize the hashtable, since you know its maximum size upon initialisation.

This creates an *exact* Dictionary since once you've added a word, you're guaranteed to be able to find it again.

■

A Bloom Filter consists of an array of (m) Boolean flags.



`https://en.wikipedia.org/wiki/Bloom_filter`

Each word is not stored in the Dictionary, but instead k different hashes are computed from it, and each of these used to set a Boolean flag in an array. When searching, compute these hashes, and if **any** of the corresponding flags are zero, you know the word cannot be in the Dictionary. If all the corresponding flags are set, then the word is **probably** in the Dictionary. You can't be certain though, because two different strings might hash to the same hash values, and the table might become congested. Such approaches are therefore known as *approximate*.

Exercise 11.5.2 Use a Bloom Filter to implement the Dictionary. Write the source files `Approx/specific.h` and `Approx/approx.c` to compile against my `dict.h` file. For simplicity, use a traditional Boolean array for the Bloom Filter (and **not** a packed array such as seen in Exercise 9.2.1).

By choosing m to be twenty times the number of words, and $k = 11$, the chances of a false match are reduced to (nearly) zero, and the tests `testdict.c` and `spelling.c` should perform in the same way as their 'exact' counterparts.

Inventing k different hash functions would be complicated, so instead we create one master hash (using Bernstein for example), and then evolve this iteratively to create the others. You could do this using something like :

```
unsigned long* _hashes(const char* s)
{
    // You'll need to free this later
    unsigned long* hashes = ncalloc(KHASHES, sizeof(unsigned long));
    // Use Bernstein from Lecture Notes (or other suitable hash)
    unsigned long bh = _hash(s);
    int ln = strlen(s);
```

```
        /* If two different strings have the same bh, then
           we need a separate way to distiguish them when using
           bh to generate a sequence */
        srand(bh*(ln*s[0] + s[ln-1]));
        unsigned long h2 = bh;
        for (int i=0; i<KHASHES; i++) {
            h2 = 33 * h2 ^ rand();
            hashes[i] = h2;
        }
        // Still need to apply modulus to these to fit table size
        return hashes;
    }
```

■

11.6 Binary Sparse Arrays

Using dynamic arrays is often a compromise between resizing the array too frequently (e.g. every time the memory required is fractionally too small) or else creating a great deal of 'spare' memory, just in case it is needed later. It is common for such dynamic arrays to double the available memory every time an index is accessed out-of-bounds.

Here we create the Binary Sparse Array (BSA) which aims to be memory efficient for small arrays, releasing new memory at an exponential rate as the array grows. Using a BSA ensures that no data is ever copied (e.g. via `remalloc`) nor too much unused memory created early. The price we pay for this memory efficient is a slightly more complex calculation for the address of the index required.

Another data structure, the Hashed Array Tree (HAT) has similar goals, but is very different in practice and requires copying of data and resizing.

www `https://en.wikipedia.org/wiki/Hashed_array_tree`

Here, the BSA structure consists of a (fixed-size) row pointer table, each cell of which enables access to a $1D$ row of data. These rows are of a known size, each of which is increasingly large as we go down the table, as shown in Figure 11.2

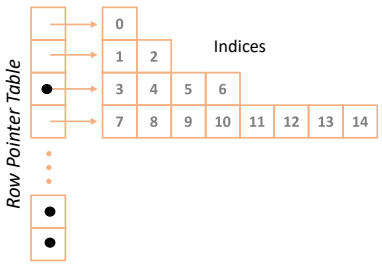


Figure 11.2: The Binary Sparse Array. A (vertical fixed-size) table allows access to each row. The rows are size 1 for the first, size 2 for the second, size 4 for third i.e. for row k it's size is 2^k.

However, the space allocation for each row is only done as-and-when required. After creating an empty BSA, and inserting values into index two and 12, then only rows one and three will have been allocated, as shown in Figure 11.3.

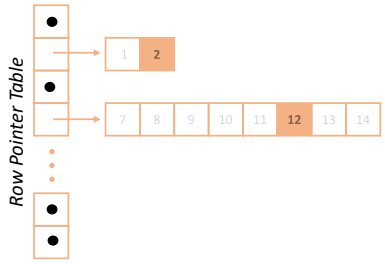


Figure 11.3: A Binary Sparse Array, showing values inserted into index 2 and 12 (shaded cells). Other rows are currently unallocated.

If the only element in a row is deleted, then the entire row is freed up, as shown in Figure 11.4 where the last element in row three is removed (index 12).

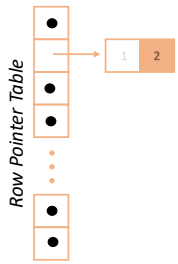


Figure 11.4: Deletion in a BSA can result in entire rows being freed up.

Exercise 11.6.1 Write an ADT to implement the `bsa.h` interface given. Write the source files `Alloc/specific.h` and `Alloc/alloc.c` such that the driver files `driver.c`,`isfactorial.c`, `sieve.c` and `fibmemo.c` work correctly. Ensure that this all works with my `Makefile` which **must be** used **unaltered**. The basic operations are :

```
bsa* bsa_init(void);
bool bsa_set(bsa* b, int indx, int d);
int* bsa_get(bsa* b, int indx);
bool bsa_delete(bsa* b, int indx);
int bsa_maxindex(bsa* b);
```

```
bool bsa_free(bsa* b);
```

The function `bsa_set()` allows an integer to be written to a particular index. If it is the first integer to be written into the row, then the space for this will need to be allocated.
`bsa_get()` returns a pointer to the integer stored in an index if it has been set, and `NULL` if it is a cell that is not in use, hasn't been written to, deleted, or past the end the maximum index used.
The functions `bsa_delete()` 'removes' an integer from an index (maybe setting a Boolean flag to show if it's in use or not), and if it was the only cell being used on that row, the entire row is freed.
Additional functionality includes :

```
bool bsa_tostring(bsa* b, char* str);
void bsa_foreach(void (*func)(int* p, int* n), bsa* b, int* acc);
```

`bsa_tostring()` allows a character based version of the BSA to be produced with each row inside `{}` brackets. The function `bsa_foreach` allows a user-defined function to be passed which is performed on each element in turn.

- The row pointer table is of a fixed size, and never gets deleted until the final call to `bsa_free()`.
- You will never use `realloc` for the BSA - rows are created as required using e.g. `calloc()` of a fixed size, and freed as required.
- Do not use the maths library in this assignment (my Makefile doesn't) - use bit manipulation if required rather than e.g. `log2()`.
- Even if you don't get all the functions to work correctly, make sure the code still compiles by writing 'dummy' functions as placeholders, even if some of the assertions fail.

This is worth 90% **of the marks.**

The manner in which you've been asked to implement the functionality above is **very** specific. However, exactly the same functionality (getting/setting etc.) could be implemented in very different ways (linked lists, 1D dynamic arrays, trees, hashing etc.). As an extension, write a 'rival' version to implement the core functionality of these functions without it necessarily being a BSA. This should still ensure that `fibmemo`, `sieve` and `isfactorial` can be compiled against it. This means you can ignore `bsa_tostring()` since this gives something very specific to the 'shape' of the BSA data structure. Again, use an unchanged version of my Makefile to achieve this. These files will be `Extension/specific.h` and `Extension/extension.c`.

If you do this, submit `extension.txt` which details what you have done, the motivation for it, and how well it works in practice (or not), and comparing it with the original BSA. This discussion (a few hundred words at most) is as (if not more) important than the code itself.
This worth 10% **of the marks.**

■

11.7 Cons, Car and Cdr

When storing real-world data, lists containing other lists are ubiquitous e.g. : `(0 (1 2) 3 4 5)`. In fact many functional (or at least partly functional) languages are based around the idea that all data are, conceptually, nested lists. Examples of such languages include Haskell and, of interest here, Lisp :

www `https://en.wikipedia.org/wiki/Lisp_(programming_language)`

One of the main operations of these languages is the ability to extract the front (head) or remainder (tail) of the list **very** efficiently. A traditional linked list is not good for this, since it would require work to untangle the head (it has a pointer to the rest of the list) and it's unclear how a list-within-a-list would be stored.

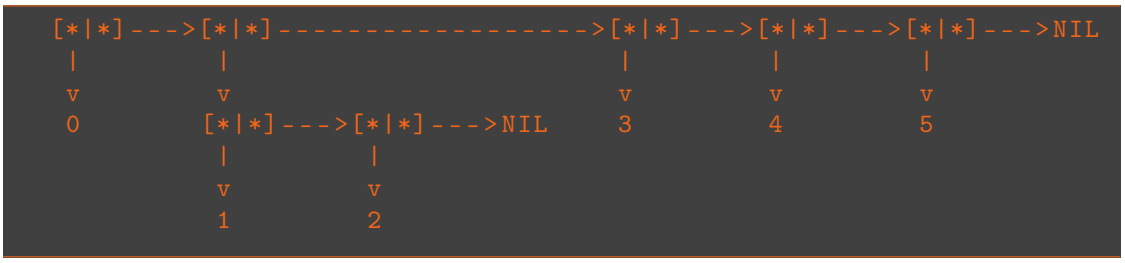
For Lisp, a better structure was developed, which is slightly more complex than a traditional linked list, but which makes the extraction of the head (and remainder) simple and fast. To encode the list `(1 2)`, which is a list containing two atoms, we use :

```
[*|*]--->[*|*]--->NIL
 |        |
 v        v
 1        2
```

Here the main struct (known as the *cons*) has two pointers. The ones going downwards known as the *car* pointers and the ones going horizontally the *cdr*[1] pointers. If we were to have a pointer p to the first *cons* of this list, the head of the list is pointed to by `p->car`, and the remainder of the list simply by `p->cdr`.

[1] From wiki: Two assembly language macros ... became the primitive operations for decomposing lists: *car* (Contents of the Address part of Register number) and *cdr* (Contents of the Decrement part of Register number).

A more complex list, `(0 (1 2) 3 4 5)` would be stored as :



The atomic elements (the integers) are stored as 'leaf' nodes with both *car* and *cdr* pointers set to NULL. The data structure used to store the atom and also *car* and *cdr* pointers is known as a *cons* (short for constructor), due the to process by which we build lists.

So, here, we're interested in recreating this data structure, allowing the user to build lists (the *cons* operation), extract the head and remainder of lists (the *car* and *cdr* operations) and other associated functions such as copying a list or counting the number of elements in it.

Exercise 11.7.1 Use a dynamic/linkedlist style approach to implement a car/cdr ADT. Write the source files `Linked/specific.h` and `Linked/linked.c`, so that they compile against my `lisp.h` and `testlisp.c` files and run successfully using my `Makefile`. The basic operations are :

```
lisp* lisp_atom(const atomtype a);
lisp* lisp_cons(const lisp* l1,  const lisp* l2);
lisp* lisp_car(const lisp* l);
lisp* lisp_cdr(const lisp* l);
atomtype lisp_getval(const lisp* l);
bool lisp_isatomic(const lisp* l);
lisp* lisp_copy(const lisp* l);
int lisp_length(const lisp* l);
void lisp_tostring(const lisp* l, char* str);
void lisp_free(lisp** l);
```

This is worth 90% of the marks. Additional functions you can implement, worth 10%, are :

```
lisp* lisp_fromstring(const char* str);
lisp* lisp_list(const int n, ...);
void lisp_reduce(void (*func)(lisp* l, atomtype* n), lisp* l, atomtype* acc);
```

and are **Extensions** worth 10% of the marks.
Even if you don't get these extensions (or other functions) to work correctly, make sure the code still compiles by writing 'dummy' functions as placeholders, even if some of the assertions fail. ■



A. House Style

A.1 Correctness

These style rules ensure your code is as-correct-as-can-be with the aid of the compiler and other tools:

FLAGS Having no warnings (or errors!) when compiling and executing with the flags:
For array bounds checking, `NULL` pointers being dereferenced, leak checking etc:

```
-Wall -Wextra -Wfloat-equal -Wvla -pedantic -std=c99
-fsanitize=undefined -fsanitize=address -g3
```

For 'final' production-ready code:

```
-Wall -Wextra -Wfloat-equal -Wvla -pedantic -std=c99
-O3
```

You can use more flags than this, obviously, but these will make sure a few of the essential warnings that commonly indicate the presence of bugs and leaks are checked. These guidelines are meant to be independent of the particular compiler used though. Sometimes it is helpful to use many compilers too, e.g. `gcc` and `clang`.
If you have unused variables (for example) in your code, it doesn't matter whether your compiler happened to tell you about it or not - it's still wrong !

BRACE Always brace all functions, `fors`, `whiles`, `if/else` etc. Somewhat controversial, this ensures that 'extra' lines tagged onto loops are dealt with correctly. For instance:

```
while(i < 10)
    printf("%i\n", i);
    i++;
```

looks like it should print out `i` 10 times, but instead runs infinitely. The programmer probably meant:

```
while(i < 10){
    printf("%i\n", i);
    i++;
}
```

GOTO You do not use any of the keywords `continue`, `goto` or `break`. The one exception is inside `switch`, where `break` is allowed because it is essential ! These keywords usually lead to tangled and complex 'spaghetti' coding style. I often recommend that you rewrite the offending code using functions, which **can** have multiple `returns` in them.

NAMES Meaningful identifiers. Make sure that functions names and variables having meaningful, but succinct, names.

REPC Repetitive code. If you've cut-and-paste large chunks of code, and made minor changes to it, you've done it wrong. Make it a function, and pass parameters that make the changes required.

```
int inbounds1(int i){
    if(i >=0 && i < MAX){
        return 1;
    }
    else{
        return 0;
    }
}

int inbounds2(int i){
    if(i >=0 && i < LEN){
        return 1;
    }
    else{
        return 0;
    }
}
```

might make more sense as:

```
int inbounds2(int i, int mx){
    if(i >=0 && i < mx){
        return 1;
    }
    else{
        return 0;
    }
}
```

GLOB No global variables. Global variables are declared 'above' `main()`, are in scope of all functions, and can be altered **anywhere** in the code. This makes it rather unclear **which** functions should be reading or writing them. You can make a case for saying that occasionally they could be useful (or better) than the alternatives, but for now, they are banned !

RETV Any functions that returns a value, should have it used:

```
scanf("%i", &i);
```

is incorrect. It returns a value that is ignored. Instead do:

```
        if(scanf("%i", &i) != 1{
            /* PANIC */
```

The only exceptions are `printf` and `putchar` which do return values but which are typically ignored.

MATCH For every `fopen` there should be a matching `fclose`. For every `malloc` there should be a `free`. This helps avoid memory leaks, when your program or functions are later used in a larger project.

STDERR When exiting your program in an error state, make sure that you `fprintf` the error on `stderr` and not `stdout`. Use `exit`, e.g.

```
if(argc != 2){
    fprintf(stderr, "Usage : %s <filename>\n", argv[0]);
    exit(EXIT_FAILURE);
}
```

A.2 Prettifying

These rules are about making your code easier to read and having a consistent style in a form that others are expecting to see.

LLEN Line length. Many people use terminal and editors that are of a fixed-width. Having excessively long lines may cause the viewer to scroll to off the screen. Keep lines short, perhaps < 60 characters. However, in a similar way to the **FLEN** rule below, it's really about the complexity of the line that's the issue, not its absolute length. A programmer would generally find:

```
bool arrcleanse(cell oldarr[HEIGHT][WIDTH], cell newarr[HEIGHT][WIDTH], int h, int w)
```

a great deal easier to read than:

```
if(a < b && j++ >= szpar(e ? true : false) || h==4){
```

despite it being twice as long.

TABS Don't use tabs to indent your code. Every editor views these differently, so you have no guarantee that I'm seeing the same layout as you do. Use spaces. This also prevents issues when cutting-and-pasting from one source to another.

INDENT Indentation: choose a style for indentation and keep to it. I happen to use 3 spaces, put opening braces for functions on a new line, but at the end of `if`,`else`, `for`, `while` etc, then close them on a new line, underneath the 'i' of the `if`:

```
int smallest(int a, int b)
{
    if(a < b){
        return a;
    }
    else{
        return b;
    }
}
```

You can use any style you like, as long as it's consistent.

MAIN The code should have function prototypes/definitions first, then `main()`, followed by the function implementation. This means the reader always know where to find `main()`, since it will be near the top of the file.

CAPS Constants are `#defined`, and use all CAPITALS. For instance:

```
#define WEEKS 52
#define MAX(a,b) (a < b ? b:a)
```

FLEN Short functions. All functions are short. It's quite difficult to put a maximum number of lines on this, but use 20 as a starting point. Exceptions include a function that simply prints a list of instructions. There would be no benefit in splitting it into smaller functions. Short functions are easier to plan, write and test.

I find it more useful to think about how hard the function is to understand, rather than its length. Therefore, a 30 line, simple function is fine, but an extremely complex and dense 15 line function might need to be split up, or more self-documentation added.

A.3 Readability

Your code should be self-documenting. Comments will be written when there is something complex to explain, and only read when something has gone catastrophically wrong. In many cases clever use of coding will avoid the need for them. The compiler never sees them, so cannot check them. If you change your code, but not your comments, they can be highly misleading.

As Kevlin Henney said :

> A common fallacy is to assume authors of incomprehensible code will somehow be able to express themselves lucidly and clearly in comments.

MAGIC No magic numbers. There should be no inexplicable numbers in your code, such as:

```
if(i < 36){
```

It's probably unclear to the reader where the 36 has come from, or what it means, even if it is obvious to the programmer at the time of writing the code. Instead, `#define` them with a meaningful name. Array overruns are often cured by being consistent with `#defines`.

BRIEF Comments are brief, and non-trivial. Worthless commenting often looks something like:

```
// Set the variable i to zero
int i = 0;
```

The programmer extracts no additional information from it. However, for more difficult edge cases, a comment might be useful.

```
// Have we reached the end of the list ?
if(t1->h == NULL){
```

To prevent lines from becoming too long, it is good practice to put comments above the line it refers to, not at the end of the same line.

TYPE You should use `typedefs`, `enums` and `structs` to increase readability.

INFIN No loops should be infinite. I'll never ask you to write a program that is meant to run forever. Therefore statements such as

```
while(1){
```

or

```
for(;;){
```

are to be avoided.

2DINDEX 2D Arrays in C are indexed `[row][col]`. Sometimes it may still work correctly, especially if you've consistently confused the two. Therefore, if you write code that indexes it `[col][row]`, or `[x][y]` it will confuse anyone else trying to understand (or reuse) your code. If you were to sketch a graph using `(i,j)` you'd almost certainly make *i* the horizontal axis, and *j* the vertical. Therefore, for any two variables it makes more sense to write `[b][a]` or `[j][i]`.